

Ex. No 1: Evidence Acquisition Using AccessData FTK Imager

Date:

Description

Forensic Toolkit or FTK is a computer forensics software product made by AccessData. This is a Windows based commercial product. For forensic investigations, the same development team has created a free version of the commercial product with fewer functionalities. This FTK Imager tool is capable of both acquiring and analyzing computer forensic evidence.

The evidence FTK Imager can acquire can be split into two main parts. They are:

- Acquiring volatile memory
- Acquiring non-volatile memory (Hard disk)

There are two possible ways this tool can be used in forensics image acquisitions:

- Using FTK Imager portable version in a USB pen drive or HDD and opening it directly from the evidence machine. This option is most frequently used in live data acquisition where the evidence PC/laptop is switched on.
- Installing FTK Imager on the investigator's laptop.

In this case the source disk should be mounted into the investigator's laptop via write blocker. The write blocker prevents data being modified in the evidence source disk while providing read-only access to the investigator's laptop. This helps to maintain the integrity of the source disk.

Acquiring volatile memory using FTK Imager

The FTK Imager tool helps investigators to collect the complete volatile memory (RAM) of a computer. The following steps will show you how to do this.

Open FTK Imager and navigate to the volatile memory icon (capture memory).

NOTE: This tool provides options to include pagefile and AD1 files when acquiring the volatile memory.

Pagefile: The pagefile (pagefile.sys) is used in Windows operating systems as volatile memory due to limitation of physical random-access memory (RAM). It is located under the “C” partition ready to use as volatile memory when the existing RAM capacity is exceeded. So this file can have quite a bit of valuable data when considering the volatile memory. Therefore, it is recommended to capture and collect this file in the acquisition.

AD1 file: AD1 is the FTK imager image file. The investigator has the option to create an AD1 file for later use.

Clicking the “capture memory” button will start acquiring the volatile memory.

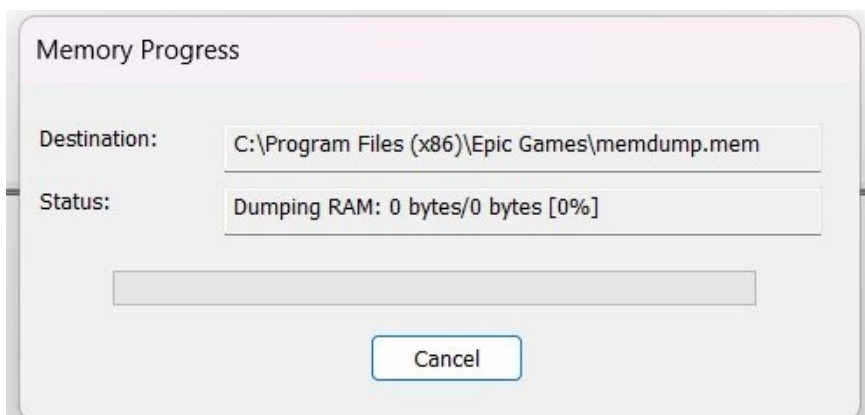


Figure 1: FTK Imager memory capture dialog for acquiring volatile memory.

NOTE: Once the acquisition has completed, the destination folder will have the acquired memory with the file extension of “.mem”.

Acquiring non-volatile memory (Disk Image) using FTK Imager

As previously stated, this same tool can be used to collect a disk image as well.

Open FTK Imager and navigate to “Create Disk Image”.

Now select the source that you need to acquire.

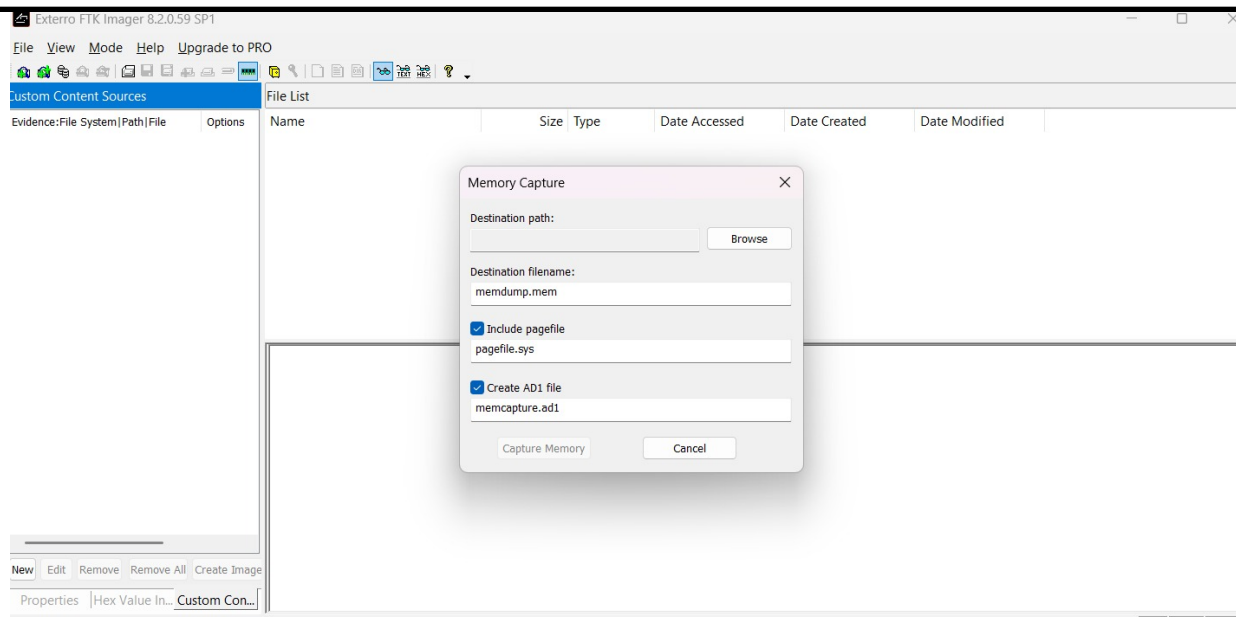


Figure 2: FTK Imager memory capture screen showing acquisition settings.

NOTE: FTK Imager is capable of acquiring physical drives (physical hard drives), logical drives (partitions), image files, contents of a folder, or CDs/DVDs. Investigators can connect external HDDs into the collection computer

via write blocker and use the “logical drive” option to select the mounted HDD as a partition.

Collecting Physical Drives

Select the “Physical Drive” option.

Select the drive you need to acquire and click “Finish”.

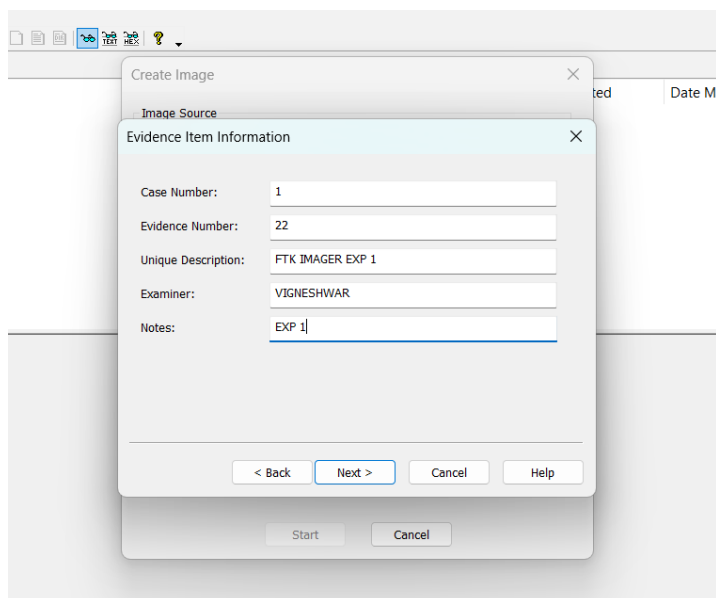


Figure 3: FTK Imager image source information screen for disk acquisition.

Raw (dd): This is the image format most commonly used by modern analysis tools. These raw file formatted images do not contain headers, metadata, or magic values. The raw format typically includes padding for any memory ranges that were intentionally skipped (i.e., device memory) or that could not be read by the acquisition tool, which helps maintain spatial integrity (relative offsets among data).

SMART: This file format is designed for Linux file systems. This format keeps the disk images as pure bitstreams with optional compression. The file consists of a standard 13-byte header followed by a series of sections. Each section includes its type string, a 64-bit offset to the next section, its 64-bit size, padding, and a CRC, in addition to actual data or comments, if applicable.

E01: this format is a proprietary format developed by Guidance Software's EnCase. This format compresses the image file. An image with this format starts with case information in the header and footer, which contains an MD5 hash of the entire bit stream. This case information contains the date and time of acquisition, examiner's name, special notes and an optional password.

AFF: Advance Forensic Format (AFF) was developed by Simson Garfinkel and Basis Technology. Its latest implementation is AFF4. The goal is to create a disk image format that does not lock the user into a proprietary format that may prevent them from being able to properly analyze it.

Now enter the case details.

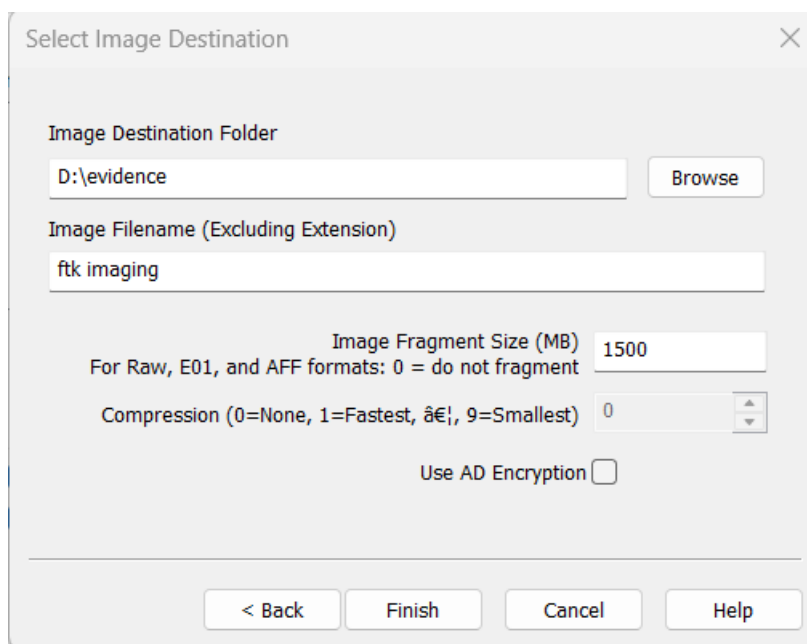


Figure 4: FTK Imager image destination settings for the forensic image.

Add an image destination (where the image file will be saved), image file name and fragment size.

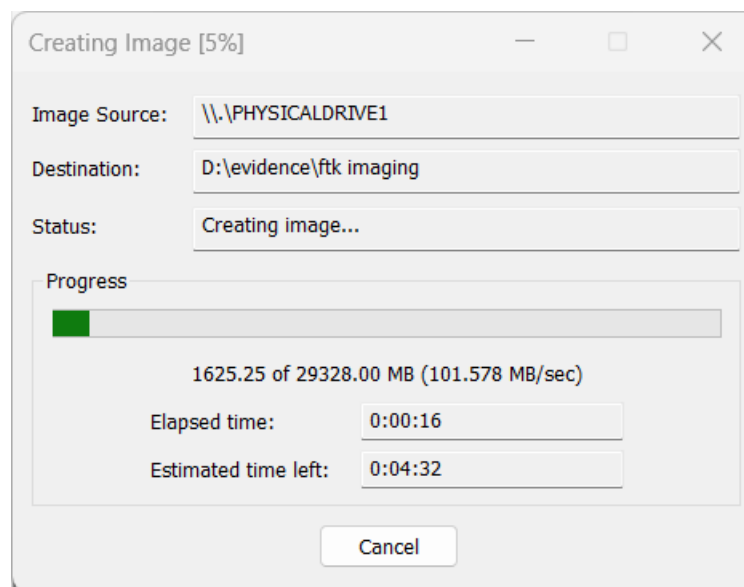


Figure 5: FTK Imager showing the progress of disk image creation.

Image Fragment Size (MB): this option will separate the image file into multiple images and save them in the same destination. If you need only one file instead of creating multiple fragmented images, you must set the image fragment size to "0".

Select the "verify images after they are created" option. This will verify the hash values once the image has created. In order to ensure integrity, it is recommended to use this option. However, this will increase the time taken to acquire your evidence, especially if you're dealing with a large disk image size.

Click "start" to start acquiring.

Once acquiring is complete, it will create a text file including all the information it has acquired.

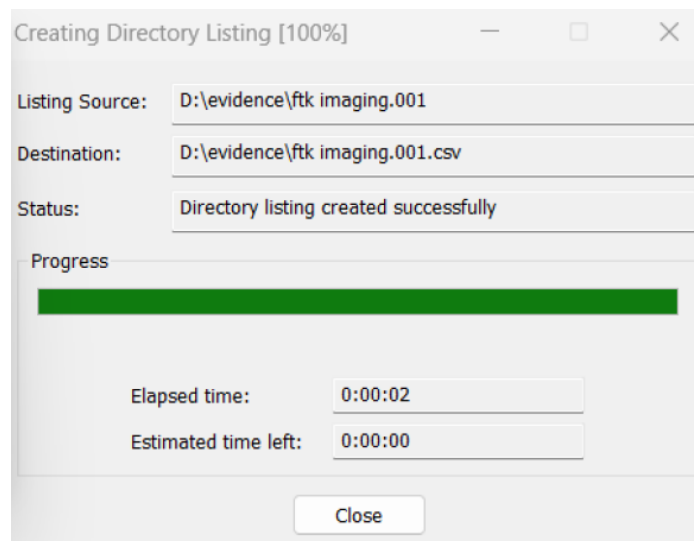


Figure 6: FTK Imager showing successful directory listing creation.

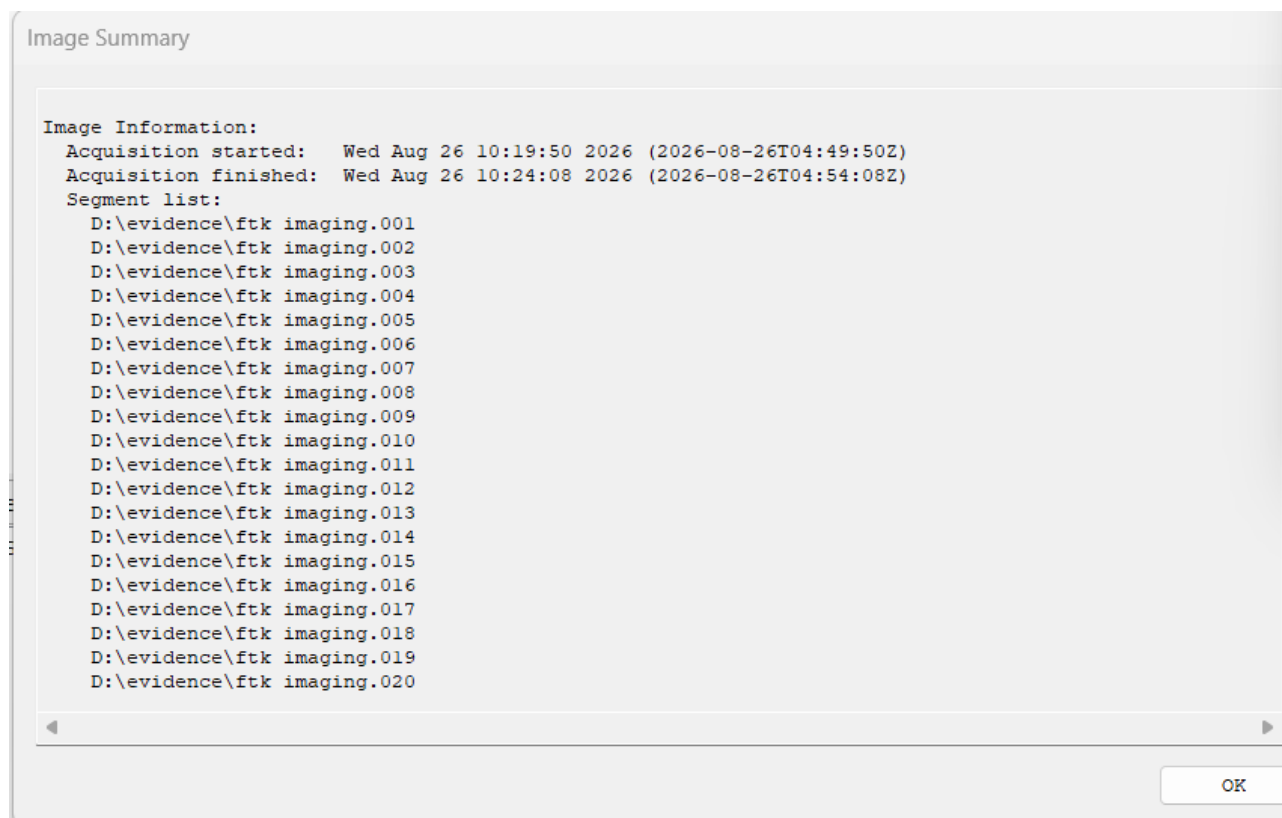


Figure 7: FTK Imager image summary showing the created image fragments.

Hash values are matched.

Ex.No:2 Recover deleted or damaged files from a storage device using Test Disk

Date:

Description:

TestDisk step by step to recover a missing partition and repair a corrupted one.

Log creation

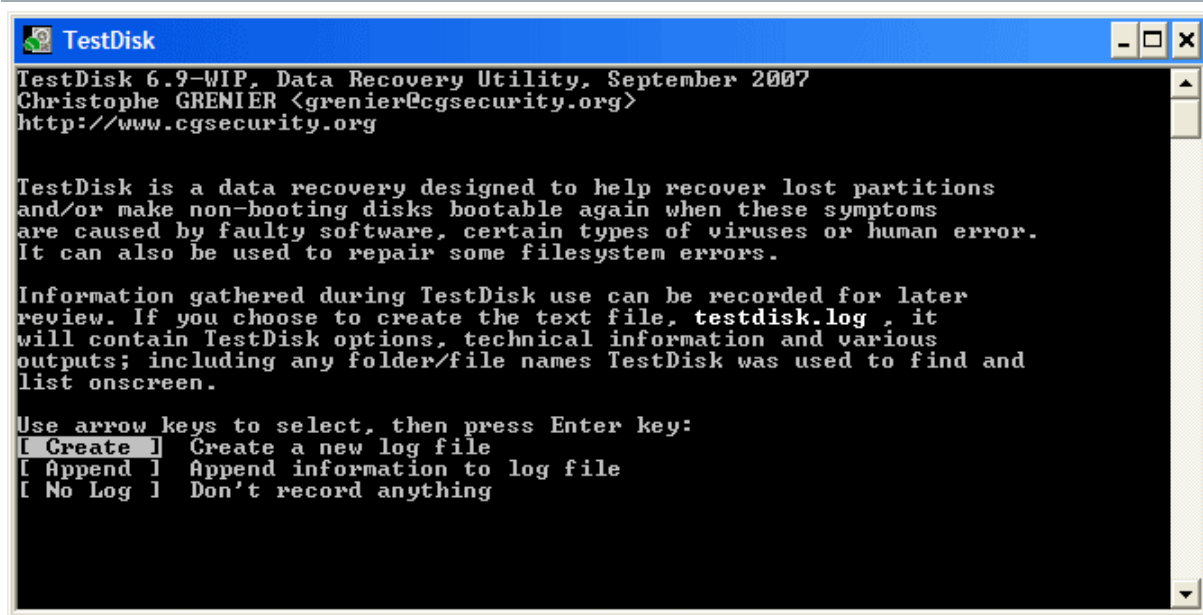


Figure 8: TestDisk log creation screen with options to create or skip a log.

- Choose Create to instruct Testdisk to create a [log file](#) containing technical information and messages, unless you have a reason to append data to the [log](#) or you execute TestDisk from read only media and must create the [log](#) elsewhere.
- Choose None if you do not want messages and details of the process to be written into a [log file](#) (useful if for example Testdisk was started from a read-only location).
- Press Enter to proceed.

Disk selection

All hard drives should be detected and listed with the correct size by TestDisk:

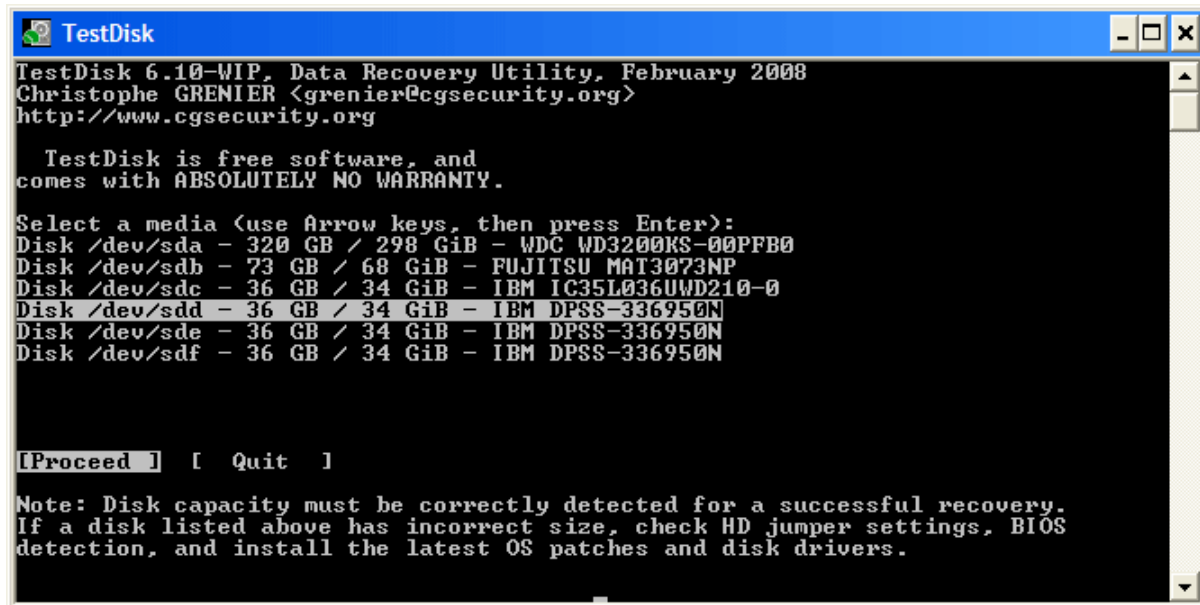


Figure 9: TestDisk disk selection screen listing available hard drives.

- Use up/down arrow keys to select your hard drive with the lost partition/s.
- Press Enter to Proceed.

X If available, use raw device `/dev/rdisk*` instead of `/dev/disk*` for faster data transfer.

Partition table type selection

TestDisk displays the partition table types.

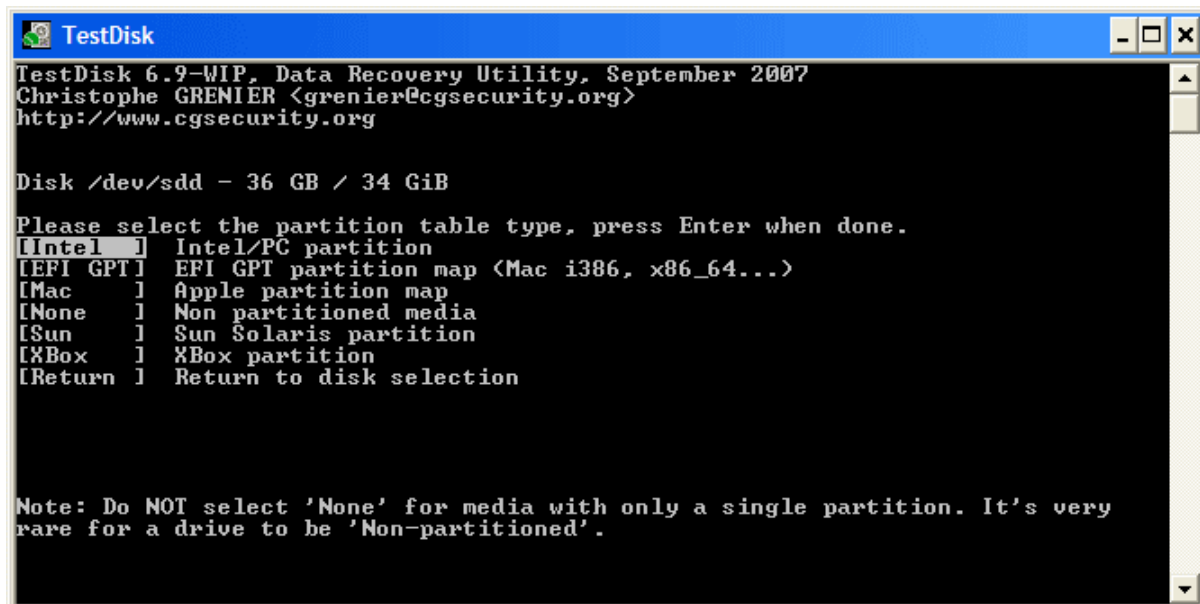


Figure 10: TestDisk partition table type selection screen.

- Select the partition table type - usually the default value is the correct one as TestDisk auto-detects the partition table type.
- Press Enter to Proceed.

Current partition table status

TestDisk displays the menus (also see [TestDisk Menu Items](#)).

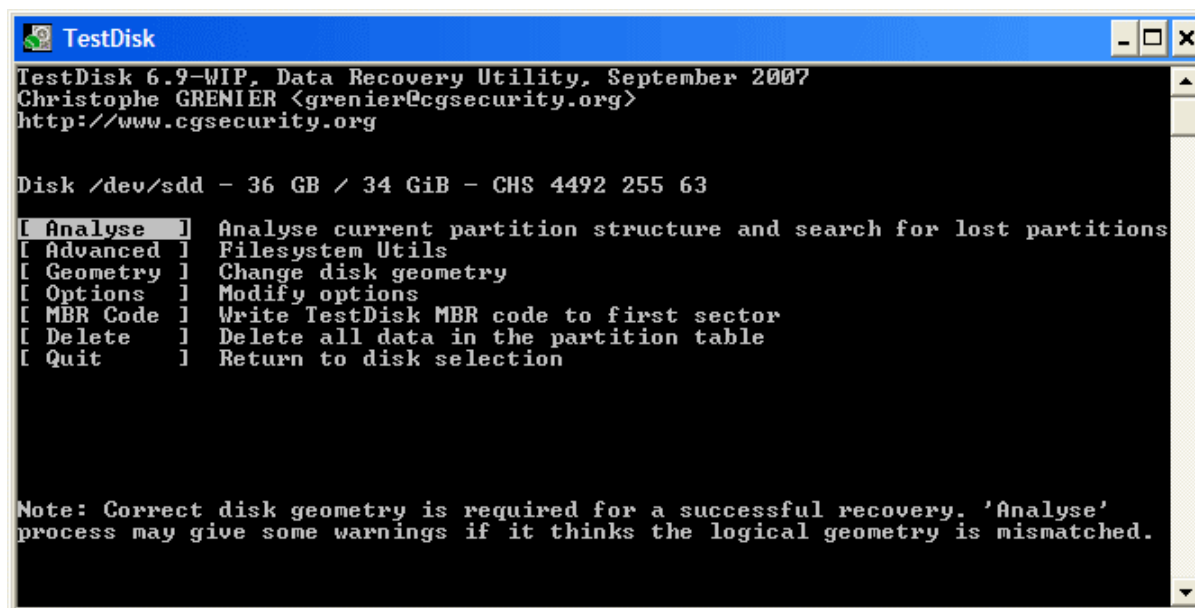


Figure 11: TestDisk Analyze screen showing the current partition structure.

- Use the default menu "Analyse" to check your current partition structure and search for lost partitions.
- Confirm at Analyse with Enter to proceed.

Now, your current partition structure is listed. Examine your current partition structure for missing partitions and errors.

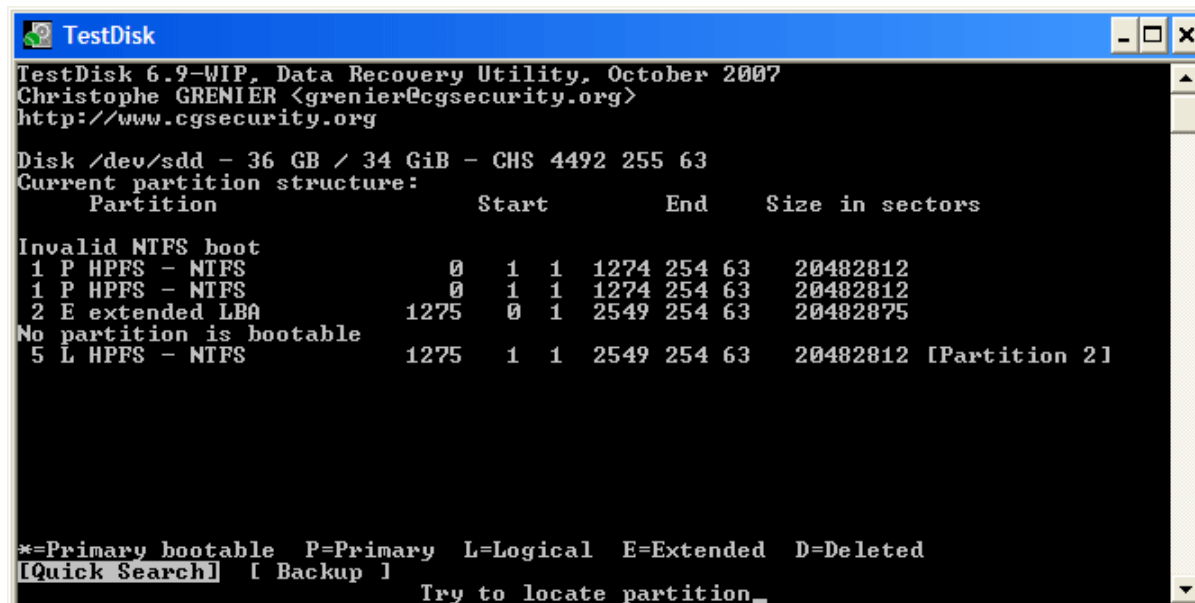


Figure 12: TestDisk Quick Search results showing detected partitions.

The first partition is listed twice which points to a corrupted partition or an invalid partition table entry.

Invalid NTFS boot points to a faulty NTFS boot sector, so it's a corrupted filesystem.

Only one logical partition (label Partition 2) is available in the extended partition. One logical partition is missing.

- Confirm at **Quick Search** to proceed.

Quick Search for partitions

TestDisk displays the first results in real time.



(click on thumb to display the image).

During the **Quick Search**, TestDisk has found two partitions including the missing logical partition labeled **Partition 3**.

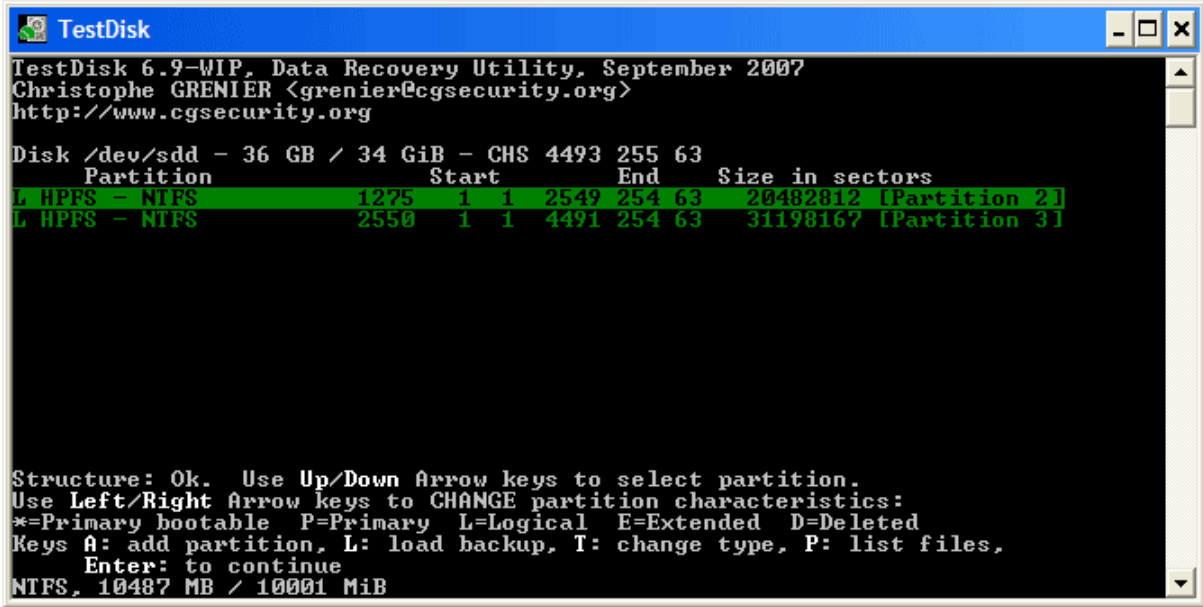


Figure 13: TestDisk Quick Search results with the missing logical partition highlighted.

- Highlight this partition and press **p** to list your files (to go back to the previous display, press q to Quit, Files listed in red are deleted entries).

All directories and data are correctly listed.

- Press Enter to proceed.

Save the partition table or search for more partitions?

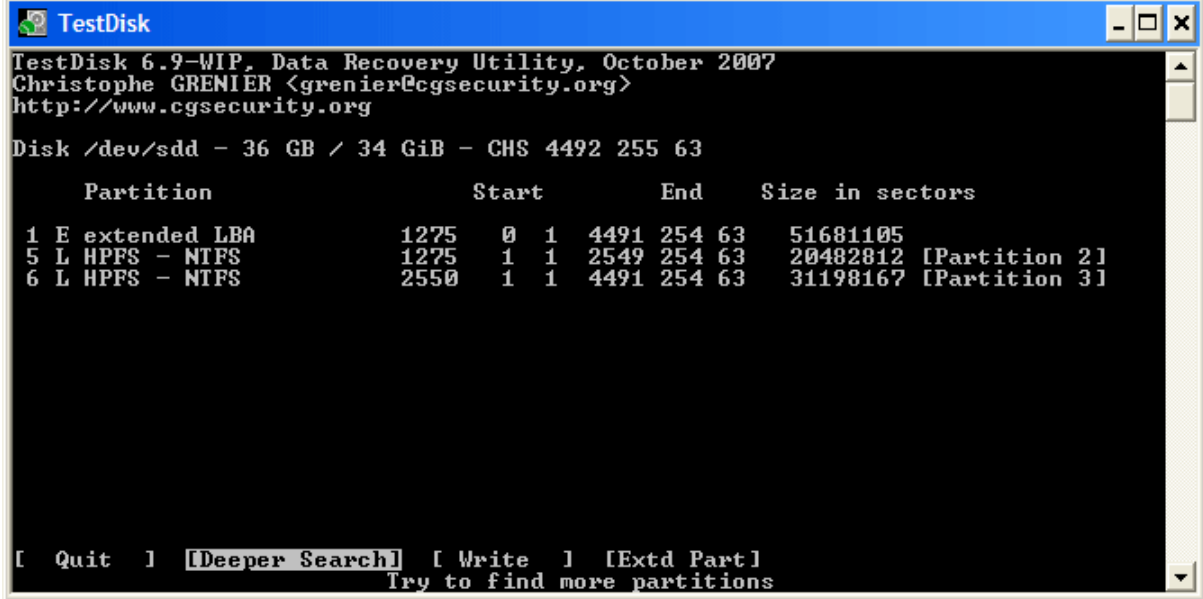


Figure 14: TestDisk screen for saving the recovered partition table or searching further.

- **When all partitions are available** and data correctly listed, you should go to the menu **Write** to save the partition structure. The menu **Extd Part** gives you the opportunity to decide if the extended partition will use all available disk space or only the required (minimal) space.
- **Since a partition, the first one, is still missing**, highlight the menu **Deeper Search** (if not done automatically already) and press Enter to proceed.

A partition is still missing: Deeper Search

Deeper Search will also search for FAT32 backup boot sector, NTFS backup boot superblock, ext2/ext3 backup superblock to detect more partitions,

it will scan each cylinder  (click on thumb).

After the Deeper Search, the results are displayed as follows:

The first partition "**Partition 1**" was found by using backup boot sector. In the last line of your display, you can read the message "**NTFS found using backup sector!**" and the size of your partition. The "partition 2" is displayed twice with different size.

Partitions listed as D(eleted) will not be recovered if you let them listed as deleted. Both partitions are listed with status **D** for deleted, because they overlap each other. You need to identify which partition to recover.

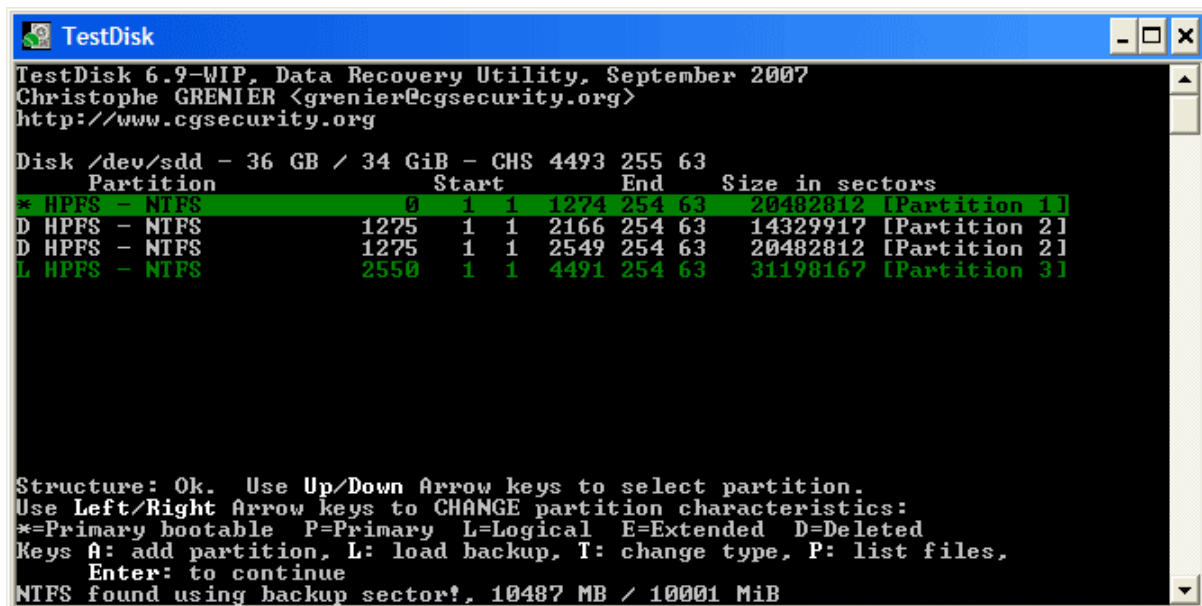


Figure 15: TestDisk Deeper Search results showing detected and overlapping partitions.

- Highlight the first partition Partition 2 and press **p** to list its data.

The file system of the upper logical partition (label Partition 2) is  (click on thumb).

- Press **q** for Quit to go back to the previous display.
- Let this partition Partition 2 with a damaged file system marked as D(deleted).
- Highlight the second partition Partition 2 below
- Press **p** to list its files.

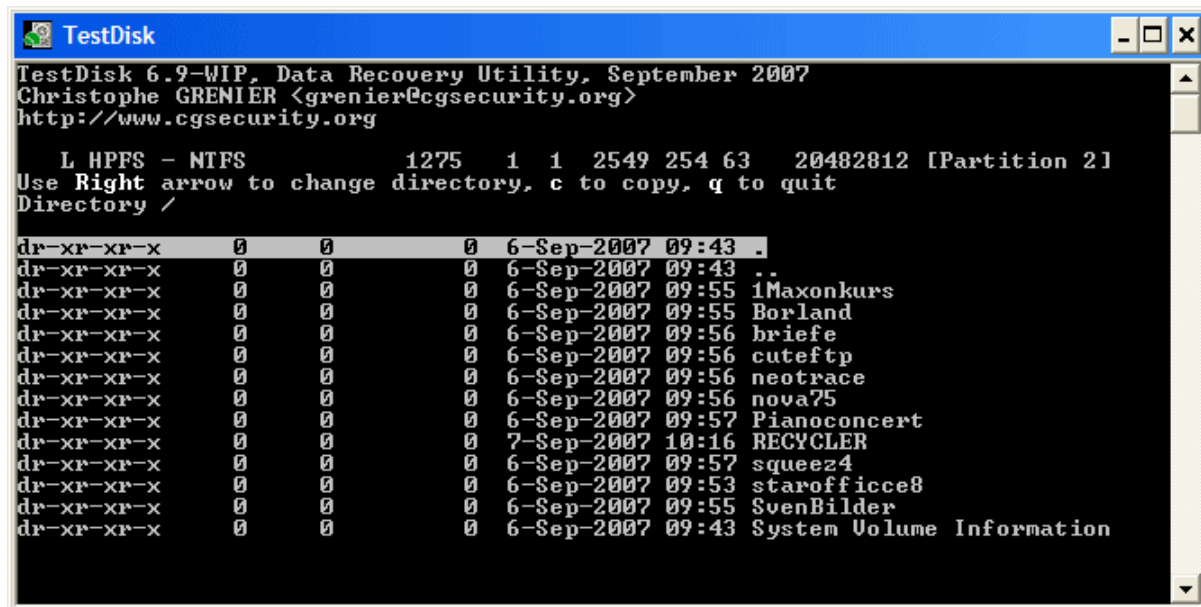


Figure 16: TestDisk file listing confirming the correct partition and recovered files.

It works, your files are listed, you have found the correct partition!

- Use the left/right arrow to navigate into your folders and watch your files for more verification

Note: FAT directory listing is limited to 10 clusters - some files may not appear but it doesn't affect recovery.

- Press q for Quit to go back to the previous display.
- The available status are Primary, * bootable, Logical and Deleted.

Using the left/right arrow keys, change the status of the selected partition from D(leted) to **L(ogical)**. This way you will be able to recover this partition.

Hint: read [How to recognize primary and logical partitions?](#)

Note: If a partition is listed *(bootable) but if you don't boot from this partition, you can change it to **P**Primary partition.

- Press Enter to proceed.

Partition table recovery

It's now possible to write the new partition structure.

Note: The extended partition is automatically set. TestDisk recognizes this using the different partition structure.

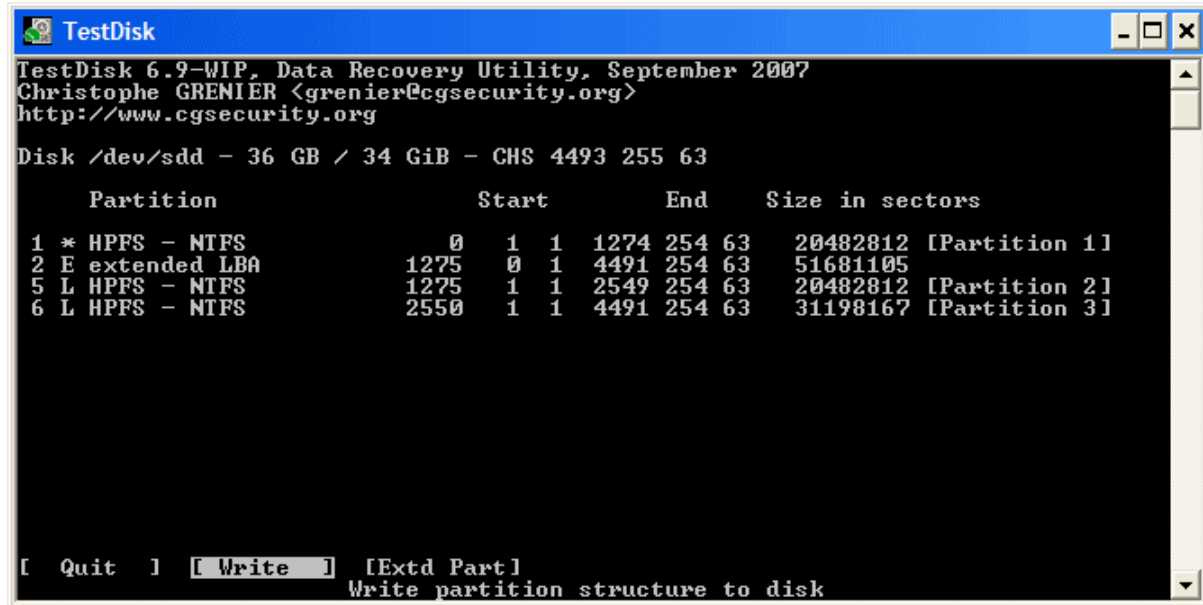


Figure 17: TestDisk partition table recovery screen with the Write option.

- If **all partitions are listed** and only in this case, confirm at **Write** with Enter, y and OK.

Now, the partitions are registered in the partition table.

NTFS Boot sector recovery

The boot sector of the first partition named Partition 1 is still damaged. It's time to fix it. The status of the NTFS boot sector is bad and the backup boot sector is valid. Boot sectors are not identical.

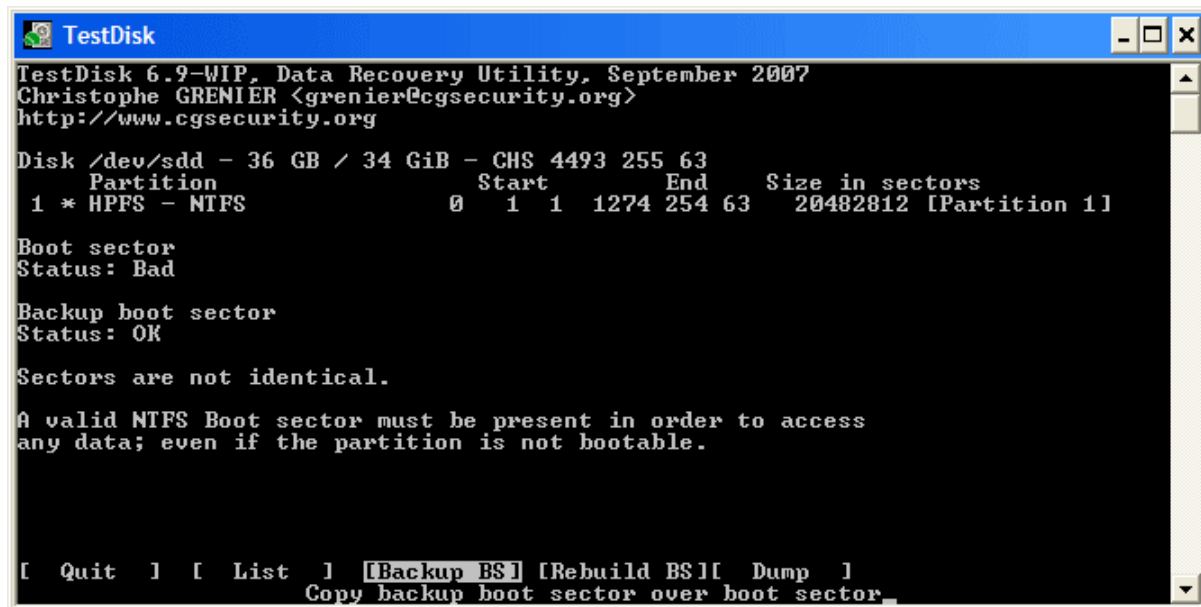


Figure 18: TestDisk NTFS boot sector recovery screen showing the damaged boot sector.

- To copy the backup of the boot sector over the boot sector, select **Backup BS**, validate with Enter, use y to confirm and next OK.

More information about repairing your boot sector under [TestDisk Menu Items](#). The following message is displayed:

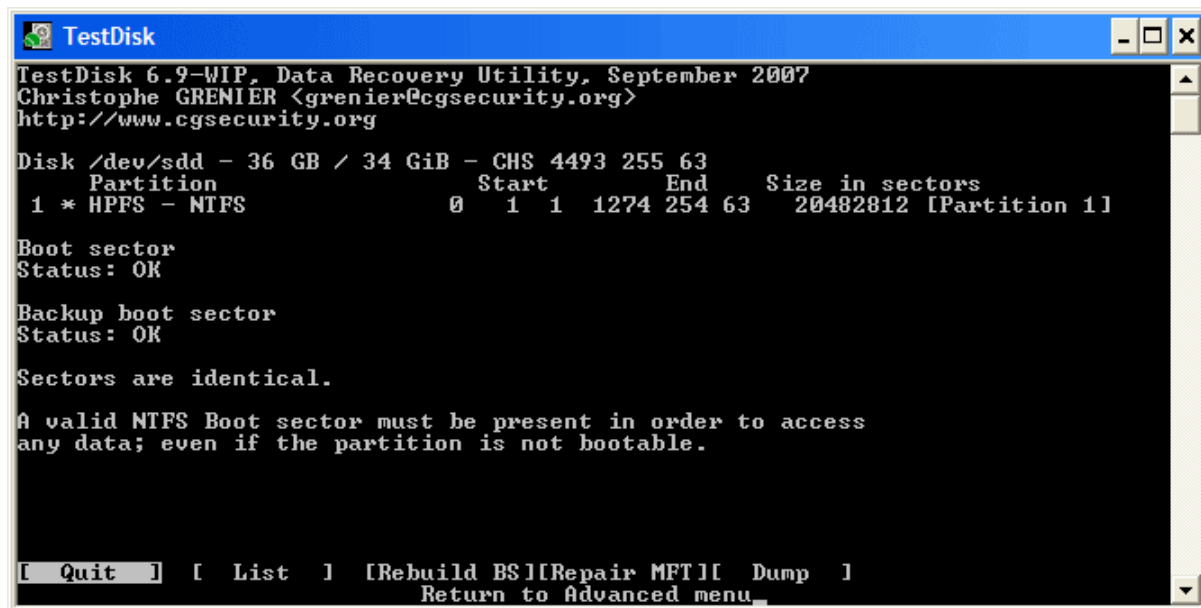


Figure 19: TestDisk screen showing the repaired boot sector and backup boot sector status.

The boot sector and its backup are now both OK and identical: the NTFS boot sector has been successfully recovered.

- Press Enter to quit.

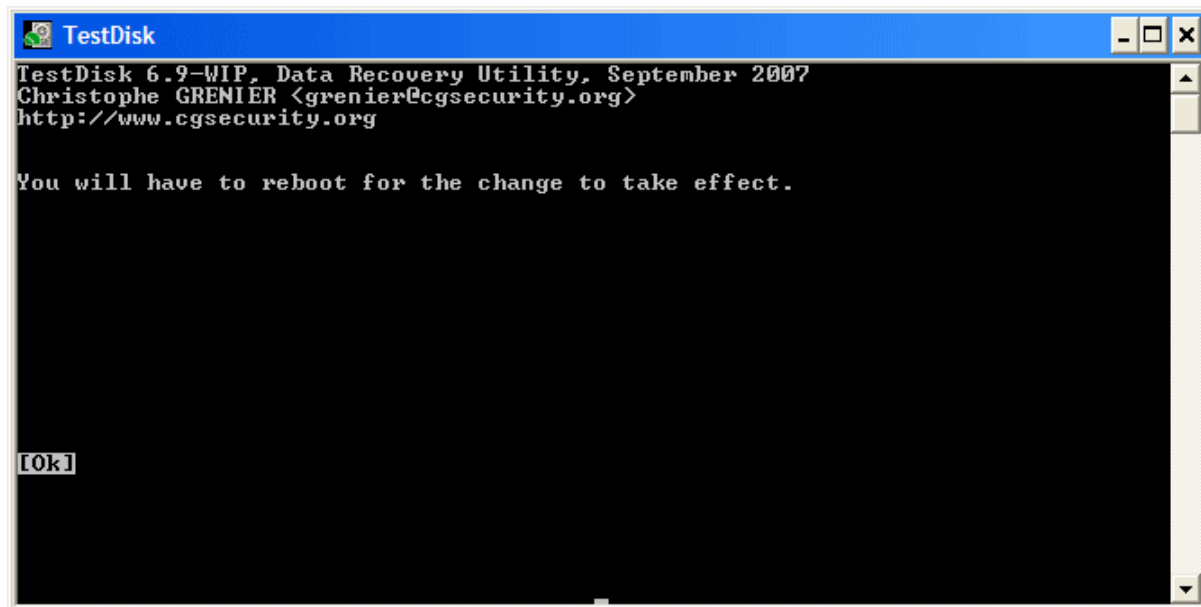


Figure 20: TestDisk restart prompt after completing partition recovery.

- TestDisk displays ***You have to restart your Computer to access your data*** so press Enter a last time and reboot your computer.

EX.NO: 3

PASSWORD CAPTURING USING WIRESHARK

Password Capturing

Wireshark can capture not only passwords but any type of information transmitted over the network: usernames, email addresses, personal information, etc. As long as we can capture network traffic, Wireshark can sniff passing passwords.

In sniffing can include passwords for various protocols such as HTTP, FTP, Telnet, etc. the captured data can be used to troubleshoot network problems, but can also be used maliciously to gain unauthorized access to sensitive information.

So, here we will see how we can capture the password using the Wireshark network capture analyzer. and see the outputs of the following steps.

Step 1: First of all, open your Wireshark tool in your window or in Linux virtual machine. and start capturing the network. suppose I am capturing my wireless fidelity.

Step 2: After starting the packet capturing we will go to the website and login the credential on that website as you can see in the image.

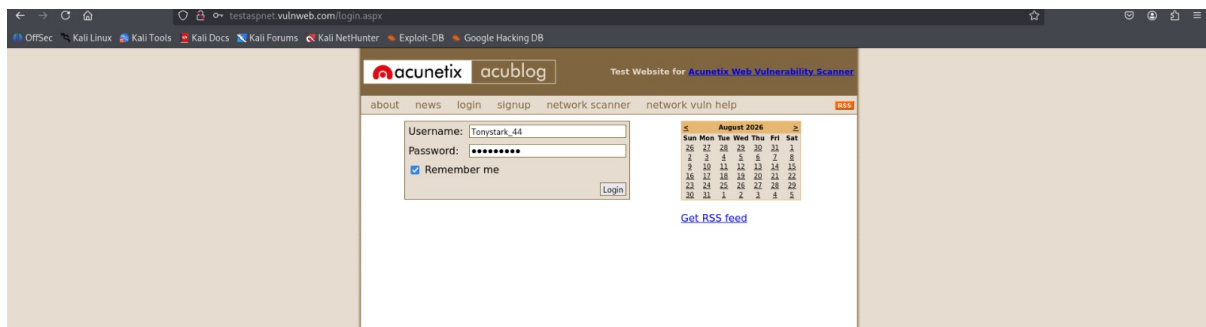


Figure 21: Example website login page used during the Wireshark packet-capture demonstration.

Step 3: Now after completing the login credential we will go and capture the password in Wireshark. for that we have to use some filter that helps to find the login credential through the packet capturing.

Step 4: Wireshark has captured some packets but we specifically looking for HTTP packets. so in the display filter bar we use some command to find all the captured HTTP packets. as you can see in the below image the green bar where we apply the filter.

http

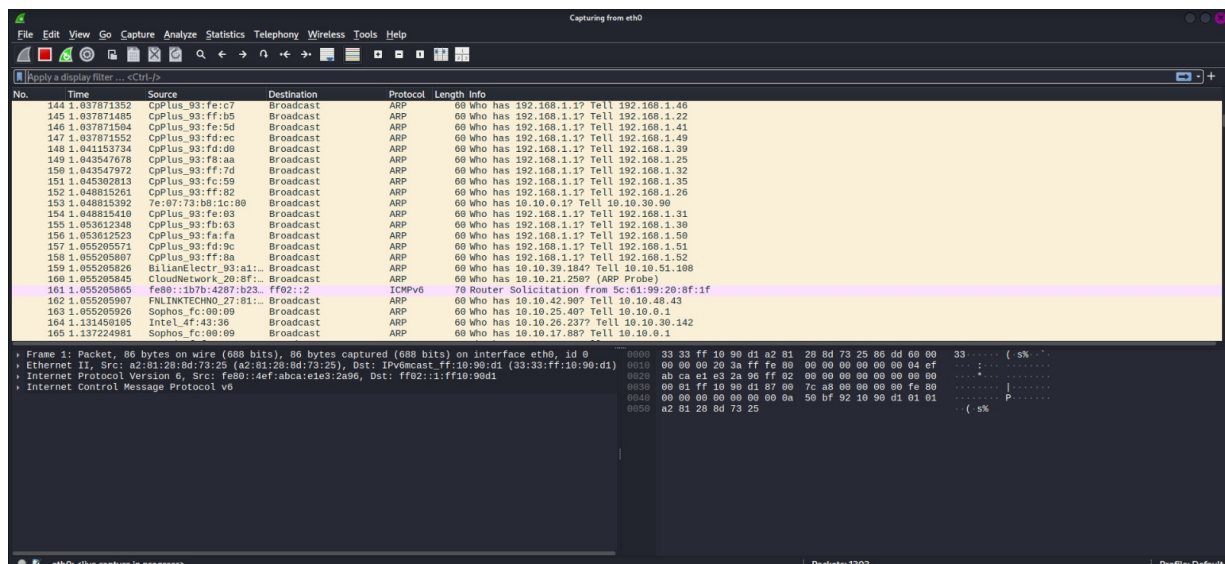


Figure 22: Wireshark packet capture showing captured network traffic and HTTP packets.

Step 5: So there are some HTTP packets are captured but we specifically looking for form data that the user submitted to the website. for that, we have a separate filter

As we know that there are main two methods used for submitting form data from web pages like login forms to the server. the methods are-

- GET
- POST

Step 6: So firstly for knowing the credential we use the first method and apply the filter for the GET methods as you can see below.

`http.request.method == "GET"`

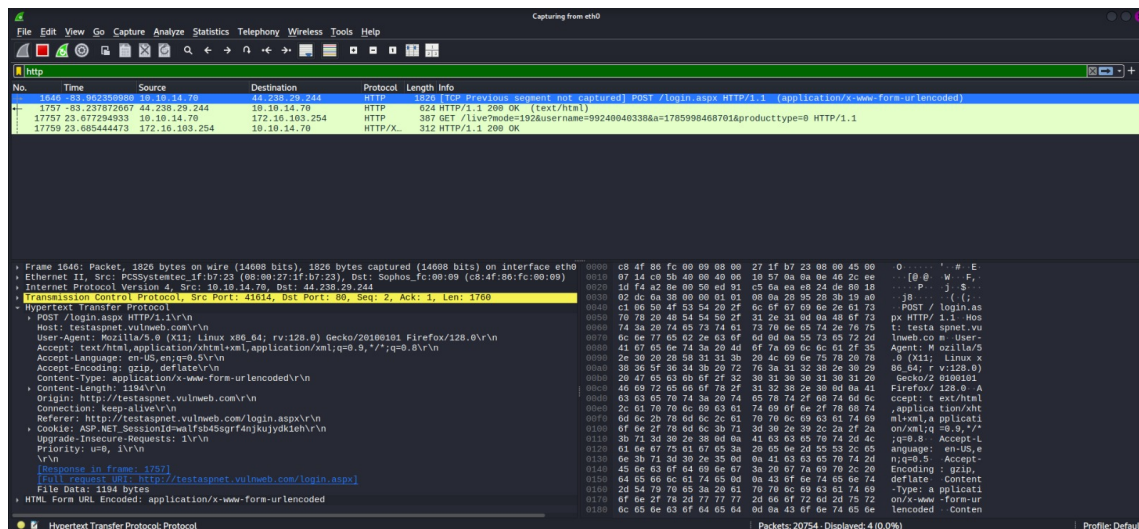


Figure 23: Wireshark GET request filter and packet details.

GET method

As you can see in the image there are two packets where the login page was requested with a GET request as well, but there is no form data submitted with a GET request.

Step 7: Now after checking the GET method if we didn't find the form data, then we will try the POST method for that we will apply the filter on Wireshark as you can see.

`http.request.method == "POST"`

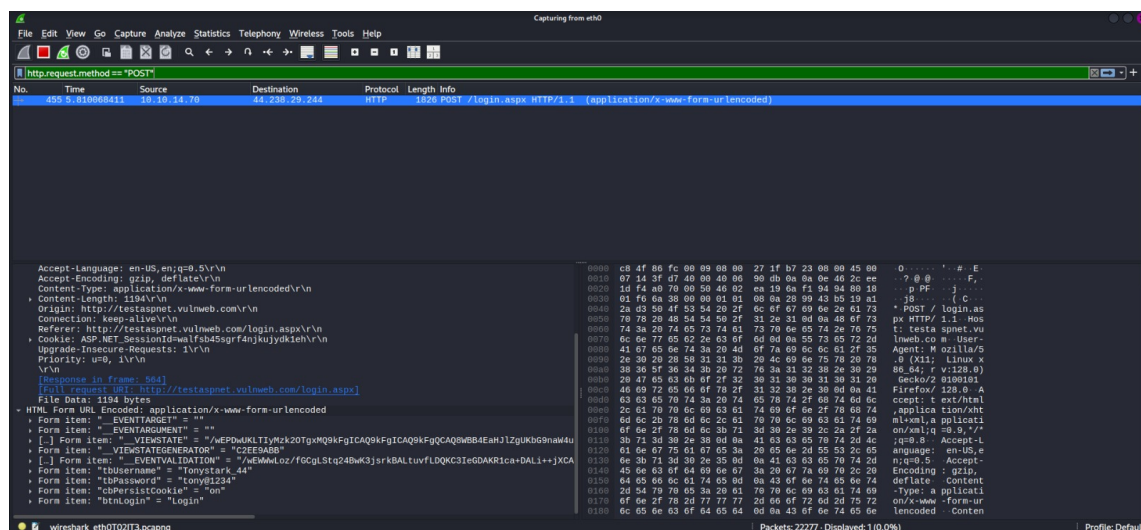


Figure 24: Wireshark POST request filter showing submitted form data.

As you can see we have a packet with form data click on the packet with user info and the application URL encoded. and click on the down-

HTML form URL Encoded where the login credential is found. login credential as it is the same that we filed on the website in step 2.

Form item: "uname" = "**Tonystark_44**"

Form item: "pass" = "**tony@1234**"

Ex. No 4: Analyze email headers and detect email spoofing using MHA (Mail Header Analyzer)

Description:

Here's a step-by-step guide on how to execute and analyze an email header:

1. Access the Email Header

- **Gmail:** Open the email, click the three dots (More) in the upper right corner, and select "Show original."
- **Outlook:** Open the email, click on "File," then "Properties," and find the "Internet headers" box.
- **Yahoo:** Open the email, click on the three dots (More), and select "View raw message."

2. Copy the Email Header

- Once you access the header, copy all the text that appears. This text contains various metadata fields about the email's journey from the sender to your inbox.

3. Identify Key Header Fields

- **From:** The sender's email address.
- **To:** The recipient's email address.
- **Date:** The date and time the email was sent.
- **Subject:** The email's subject line.
- **Return-Path:** The return address if the email bounces.
- **Received:** A list of servers the email passed through on its way to the recipient. This is usually the most critical part of header analysis.
- **Message-ID:** A unique identifier for the email.
- **SPF (Sender Policy Framework) / DKIM (DomainKeys Identified Mail):** Indicators of email authentication.

4. Analyze the 'Received' Fields

- The Received fields show the path the email took from the sender to the recipient. These fields are listed in reverse order (from the last server to the first).
- Each Received line will show:
 - **The sending server's hostname/IP address.**
 - **The receiving server's hostname/IP address.**
 - **The date and time the email passed through.**

5. Check for IP Addresses and Hostnames

- Use tools like **WHOIS** or online IP lookup services to identify the geographical location and ownership of the IP addresses found in the Received lines.
- Check if any IP addresses are suspicious or if the hostname does not match the expected sending server.

6. Examine the SPF, DKIM, and DMARC Results

- **SPF:** Checks if the sender's IP is authorized to send emails for the domain.
 - **Pass:** Indicates the IP is authorized.
 - **Fail:** The IP is not authorized and could be a sign of spoofing.
- **DKIM:** Verifies that the email content has not been altered in transit.
 - **Pass:** The email has not been tampered with.
 - **Fail:** The email content may have been altered.
- **DMARC:** A policy that uses SPF and DKIM to authenticate the email.
 - **Pass:** Indicates alignment with SPF/DKIM policies.
 - **Fail:** Indicates possible spoofing or tampering.

7. Analyze the 'Message-ID'

- The Message-ID is unique to each email. It usually includes the domain of the sending server.
- Check if the domain in the Message-ID matches the sender's domain. If not, it could be a sign of spoofing.

8. Look for Anomalies

- **Mismatch in Email Domains:** Check if the From domain matches the domains in Return-Path, Message-ID, and other authentication results.
- **Strange IP Addresses:** Be wary of IP addresses that do not match the known sending server or are from unexpected locations.
- **Suspicious Time Stamps:** Check for significant time discrepancies in the Received lines, which could indicate delays or rerouting.

9. Use Online Tools

- Use online email header analysis tools like **MXToolbox** or **Google's G Suite Toolbox** to automatically parse and analyze the email header.

10. Document and Report Findings

- Document all your findings, especially if you detect phishing or spoofing.
- Report suspicious emails to your IT department or email provider for further investigation.

Example of a Header Analysis

Here's a quick example with a sample header:

vbnet

Copy code

Received: from mail.example.com (mail.example.com [192.0.2.1])

by mail.receiver.com with ESMTP id u29si8604336pjs.40.2023.08.10.07.00.16;

Thu, 10 Aug 2023 07:00:16 -0700 (PDT)

Received: by mail.example.com with SMTP id a1mr1243772ywh.51;

Thu, 10 Aug 2023 07:00:15 -0700 (PDT)

Message-ID: <CA+7eu=4pSeXgQ@mail.example.com>

- **Check IP Addresses:** Lookup 192.0.2.1 and ensure it is associated with mail.example.com.
- **Verify SPF/DKIM:** Check if mail.example.com is authorized to send emails for this domain.
- **Compare Time Stamps:** Ensure the times make sense chronologically without unusual delays.

Output & Demonstration:

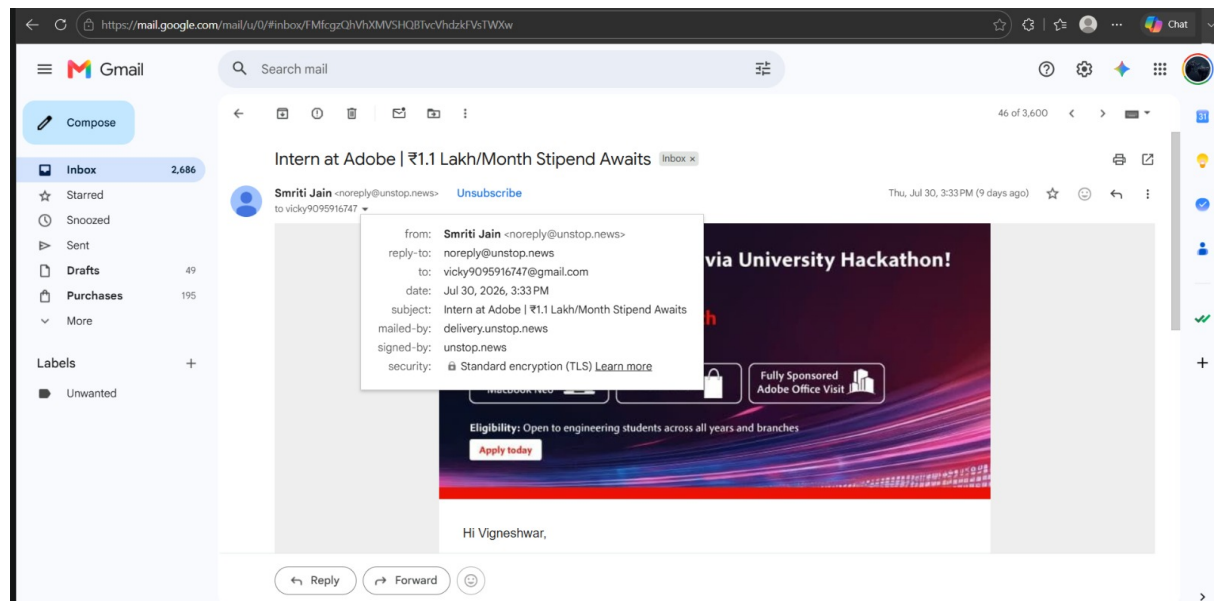


Figure 25: Gmail message view showing the sample email used for header analysis.

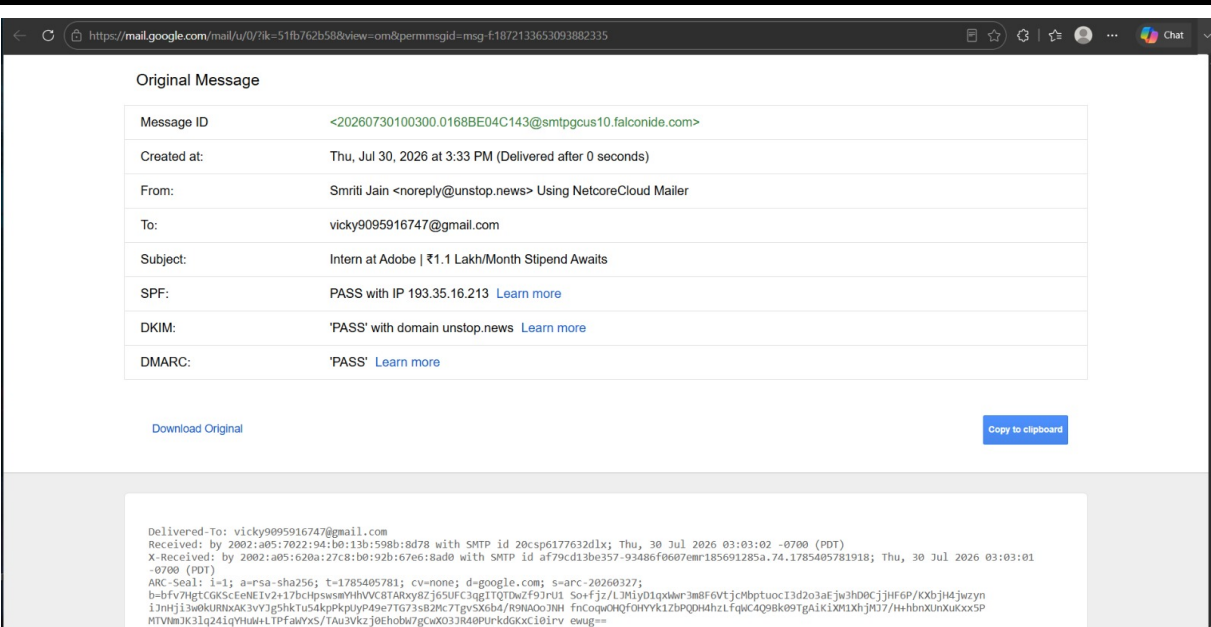


Figure 26: Gmail original-message view displaying email header information.

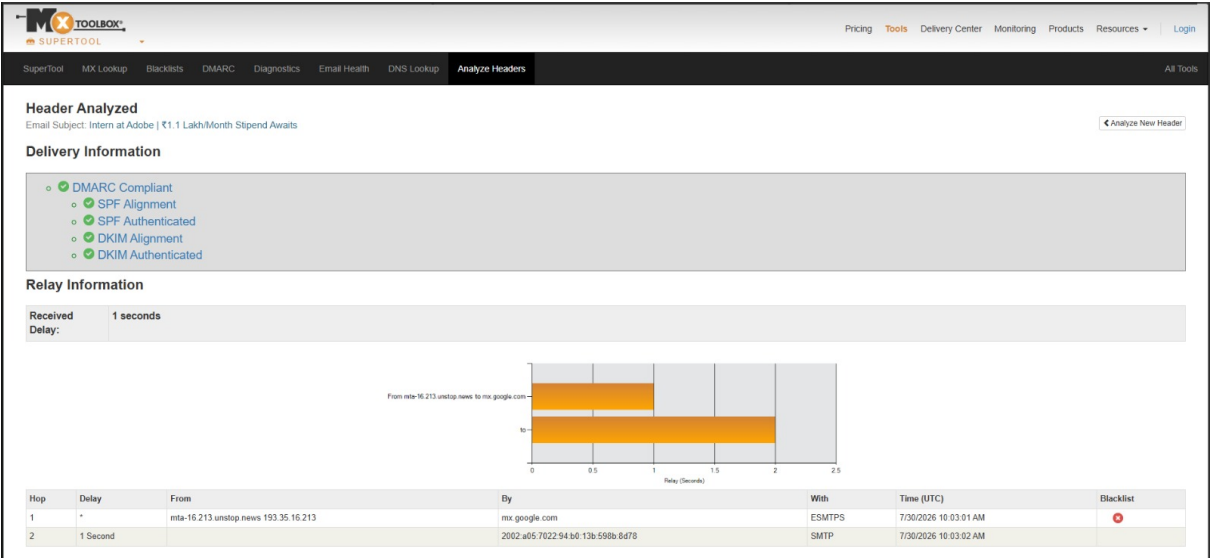


Figure 27: MXToolbox Header Analyzer results showing delivery and relay information.

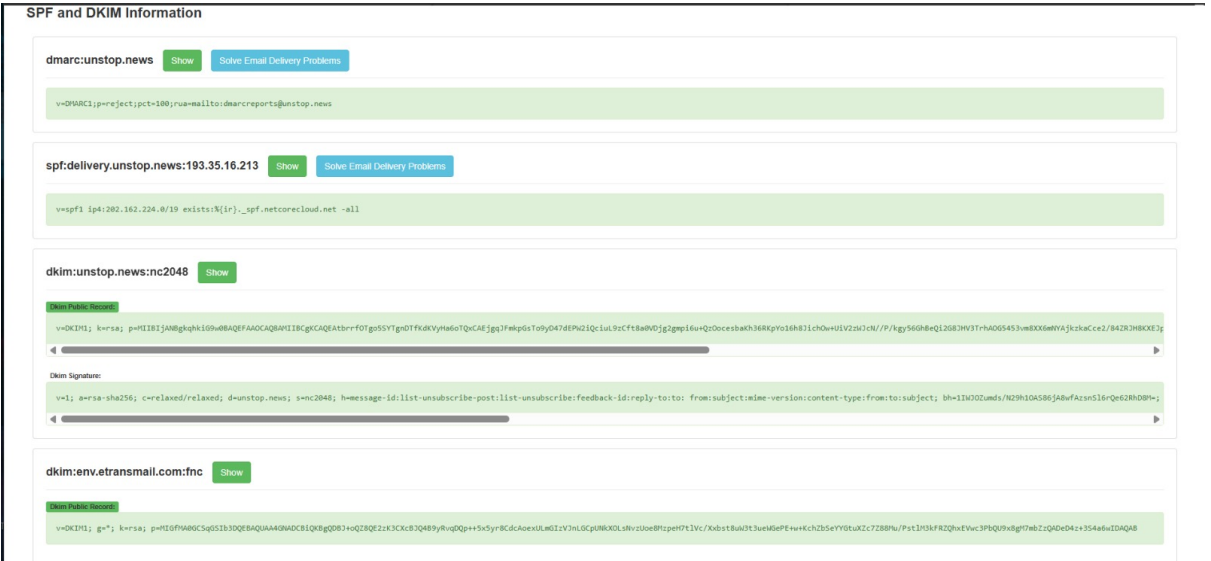


Figure 28: Header analysis results showing SPF and DKIM information.

Ex. No 5 Use Autopsy to create a case and import evidence

Description

Autopsy is an open-source digital forensics platform used for analyzing and extracting data from digital devices. Here's a step-by-step process on how to use Autopsy for a basic forensic investigation:

Drive link: https://drive.google.com/drive/u/1/folders/1ilSFY7Tqn2L7AjQGhq8yJ8kixc_xTU-v

Download files: [4Dell Latitude CPi.E01](#), [4Dell Latitude CPi.E02](#)

1. Installation

- **Download and Install:** Autopsy can be downloaded from the official website. Follow the installation instructions based on your operating system (Windows, Linux, or macOS).

2. Starting a New Case

- **Open Autopsy:** Launch the application after installation.
- **Create a New Case:**
 - Click on New Case.
 - Enter the case name and location where the case data will be stored.
 - Fill in the details like the case number, examiner's name, etc., and click Next.

3. Adding a Data Source

- **Choose the Type of Data Source:**
 - After creating a case, you'll be prompted to add a data source.
 - You can add disk images, directories, logical files, or local disks.
- **Select the Data Source:**
 - Browse to the location of the image file (e.g., .E01, .dd, or .raw), physical disk, or directory you want to analyze.
 - https://drive.google.com/drive/u/1/folders/1ilSFY7Tqn2L7AjQGhq8yJ8kixc_xTU-v
- **Configure Ingest Modules:**
 - Autopsy allows you to select specific analysis modules such as File Type Identification, Keyword Search, Hash Lookup, etc.
 - You can enable or disable these based on your investigation needs.
- **Start Analysis:** Click Next to start the analysis.

4. Initial Analysis and Overview

- **Ingest Progress:** As Autopsy processes the data source, you'll see the progress in the lower-left corner.
- **Explore the Resulting Artifacts:**
 - Autopsy automatically categorizes findings such as web artifacts, file system metadata, and communication records.
- **Use the Tree Viewer:**
 - On the left pane, you'll see a tree structure where you can explore different aspects like File System, Web History, Email, etc.

5. Detailed Analysis

- **Keyword Search:**
 - You can perform specific keyword searches using the Keyword Search module.
 - Use pre-configured lists or enter custom keywords.
- **File Analysis:**
 - Navigate through files and folders under the File Types or File System section.
 - Open, view, or export files for further examination.
- **Timeline Analysis:**
 - Use the Timeline module to visualize events based on timestamps.
 - This can help track user activity over time.
- **Hash Analysis:**
 - Compare file hashes with known databases to identify known good or bad files.

6. Reporting

- **Generate a Report:**
 - After analyzing the data, click on Generate Report from the toolbar.
 - Choose the type of report (HTML, CSV, Excel, etc.).
 - Select which parts of the analysis you want to include in the report.
- **Export Findings:**
 - Export individual files or artifacts that you need for your report or further analysis.
- **Final Review:**
 - Review the report to ensure it includes all relevant information.
 - Save or print the report for use in your case.

7. Case Closure

- **Close the Case:**
 - Once you have completed your investigation, close the case within Autopsy.
- **Archiving:**

- Ensure all data and reports are properly archived according to your organization's policies.

8. Advanced Features (Optional)

- **Custom Ingest Modules:**
 - Autopsy allows for custom modules to be added if you need specific analysis tools not included by default.
- **Collaboration:**
 - Autopsy can be configured for multi-user cases if you're working in a team environment.

Output & Demonstration:

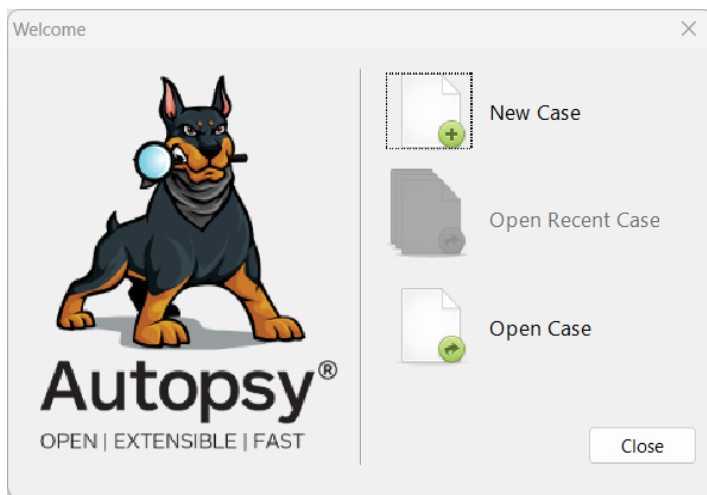


Figure 32: Autopsy welcome screen with options to create or open a case.

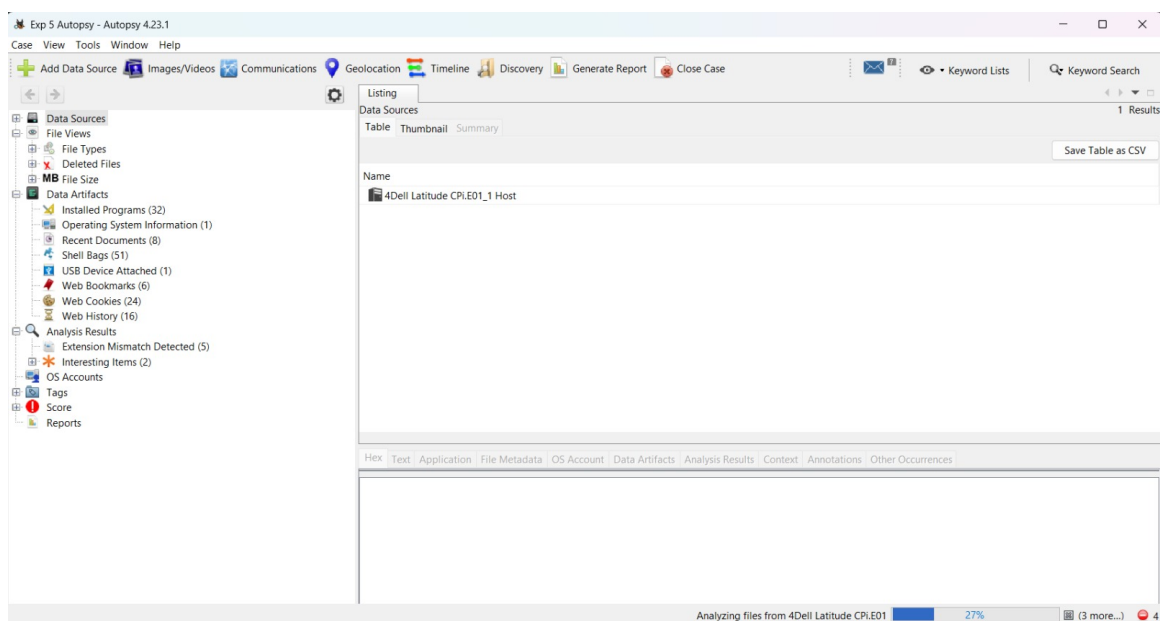


Figure 33: Autopsy case interface showing the case data and analysis workspace.

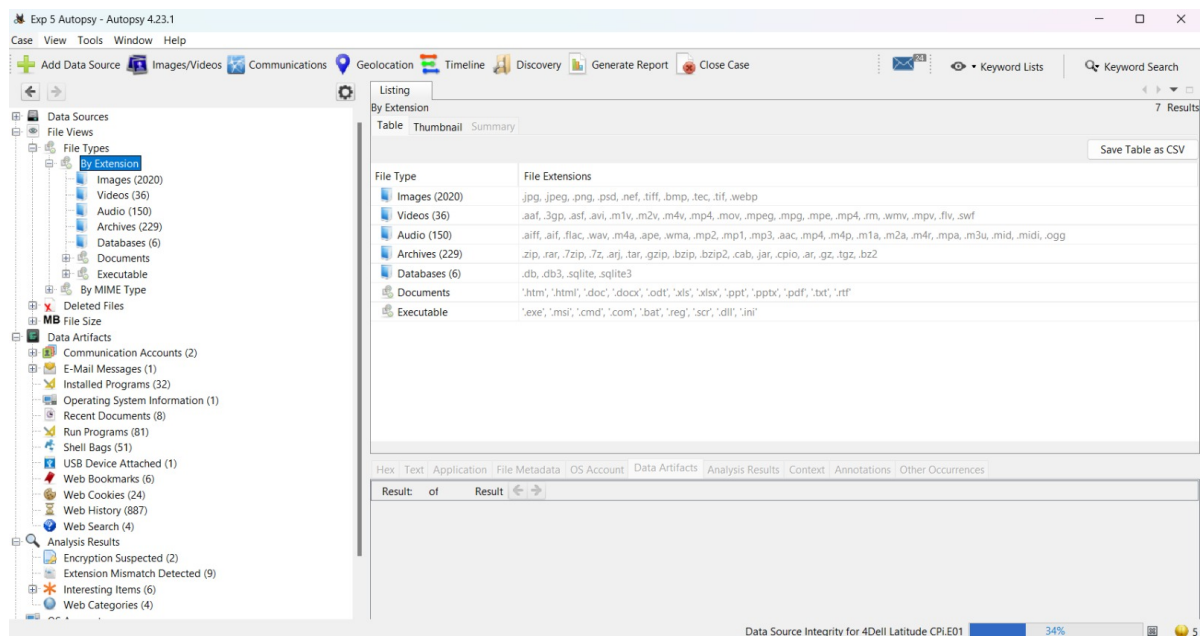


Figure 34: Autopsy analysis interface showing the evidence tree and selected artifacts.

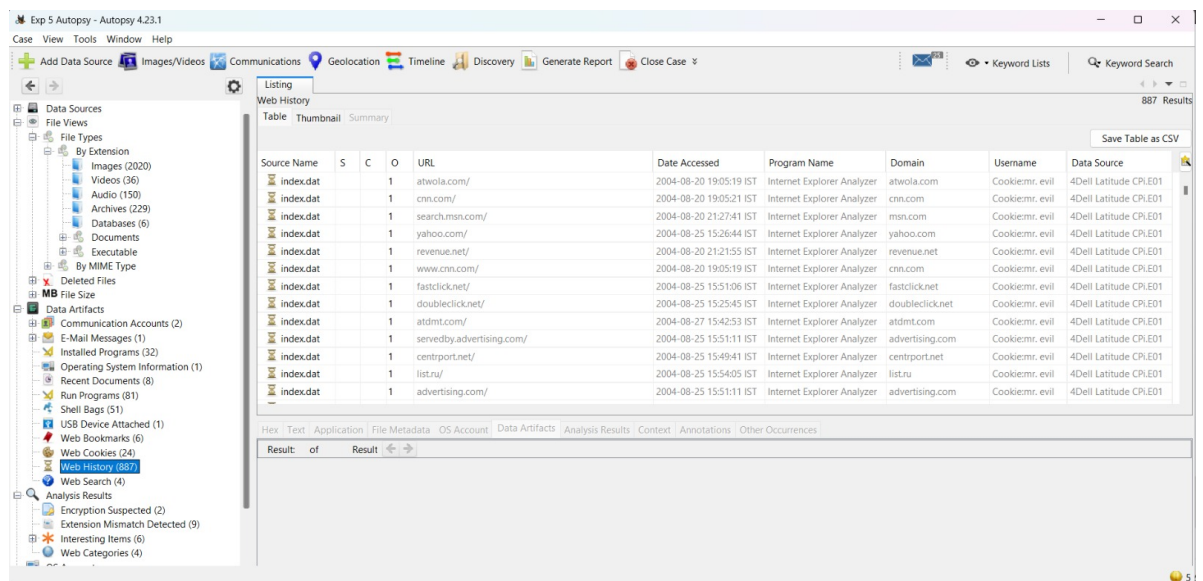


Figure 35: Autopsy file-analysis view listing files and their metadata.

Ex. No:6 Use Sleuth Kit to analyze digital evidence

Objective

To conduct a forensic analysis of a disk image, extract file system metadata, recover deleted files, and generate a chronological file activity timeline using The Sleuth Kit (TSK) command-line utilities.

Description

The Sleuth Kit (TSK) is a collection of command-line tools that allow you to analyze disk images and recover digital evidence. Here's a step-by-step guide to using Sleuth Kit on a Windows machine to analyze digital evidence:

Step 1: Install Sleuth Kit

1. Download Sleuth Kit:

- Visit the official [Sleuth Kit website](https://www.sleuthkit.org/) or https://drive.google.com/drive/u/1/folders/1ilSFY7Tqn2L7AjQGhq8yJ8kixc_xTU-v
- and download the latest version for Windows.

2. Install Sleuth Kit:

- Run the installer and follow the instructions to install Sleuth Kit on your Windows machine.

Step 2: Acquire the Disk Image

Before analysis, you need a disk image of the evidence. This can be an image of a hard drive, memory card, or any other storage device.

1. Create Disk Image:

- Use a tool like FTK Imager or dd to create a bit-by-bit copy of the storage device. Ensure the image is in a format supported by Sleuth Kit, such as .dd, .raw, .img, or .E01.
- Download the below files from the google drive
 1. 4Dell Latitude CPi.E01
 2. 4Dell Latitude CPi.E02

Step 3: Mount the Disk Image (Optional)

Mounting the disk image makes it easier to analyze the file system.

1. Mount the Image:

- You can use a tool like OSFMount to mount the image as a virtual drive on your Windows system.
- This step is optional but helps with navigating the file system easily.

Step 4: Analyze the File System

Use Sleuth Kit tools to analyze the file system and locate evidence.

1. Navigate to the Sleuth Kit Directory:

- Open the Command Prompt and navigate to the directory where Sleuth Kit is installed.
- 2. **Identify File System Type with fsstat:**
 - Run the command:
arduino
fsstat [image file] > filesystem_info.txt
 - This command outputs information about the file system, which is crucial for understanding the structure of the disk.
- 3. **List Partitions with mmls:**
 - Run the command:
arduino
mmls [image file] > partitions.txt
 - This command lists the partitions within the image file.
- 4. **Analyze File System with fls:**
 - Run the command:
arduino
fls -r [image file] > file_list.txt
 - This command recursively lists files and directories in the file system, showing their metadata.
- 5. **Recover Deleted Files with icat:**
 - To extract a specific file, use icat:
css
icat [image file] [inode number] > [output file]
 - Replace [inode number] with the inode of the file you want to recover, which you can find from the fls output.

Step 5: Analyze Metadata

Extract metadata from files to understand more about the file's history.

1. **View Metadata with istat:**
 - Run the command:
css
istat [image file] [inode number] > metadata_info.txt
 - This provides detailed information about a file, including timestamps, size, and allocation status.

Step 6: Timeline Analysis (Optional)

Creating a timeline of file activity can be crucial in an investigation.

1. **Create Timeline with mactime:**
 - Generate a body file using fls:

```
css
```

```
fls -m / -r [image file] > body.txt
```

- Then create the timeline:

```
css
```

```
mactime -b body.txt > timeline.txt
```

- The timeline includes MAC (Modified, Accessed, Changed) times of files.

Step 7: Generate a Report

Document your findings by compiling all the data into a comprehensive report.

1. Compile the Data:

- Gather all the output files (e.g., filesystem_info.txt, partitions.txt, file_list.txt, metadata_info.txt, timeline.txt).

2. Analyze and Document:

- Review the findings, highlight important evidence, and write a report summarizing the investigation's results.

Step 8: Finalize and Store Evidence

Ensure that all evidence and reports are securely stored.

1. Archive Evidence:

- Use a secure method to archive the disk image and analysis results, ensuring integrity and confidentiality.

2. Store Securely:

- Store the archived data in a secure location, following the chain of custody procedures.

By following these steps, you can use Sleuth Kit on a Windows machine to effectively analyze digital evidence and extract crucial information for your investigation.

Output:

```
C:\Users\vicky\SleuthKit> mmls.exe "D:\down\4Dell Latitude CPi.E01"
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

    Slot    Start      End      Length    Description
000:  Meta    0000000000  0000000000  0000000001  Primary Table (#0)
001:  -----  0000000000  0000000062  0000000063  Unallocated
002:  000:000  0000000063  0009510479  0009510417  NTFS / exFAT (0x07)
003:  -----  0009510480  0009514259  0000003780  Unallocated

C:\Users\vicky\SleuthKit>
```

Figure 36: Command prompt displaying the DOS partition table layout using Sleuth Kit's mmls tool.

```

C:\Users\vicky\SleuthKit> fsstat.exe -o 63 "D:\down\4Dell Latitude CPi.E01"
FILE SYSTEM INFORMATION
-----
File System Type: NTFS
Volume Serial Number: B26CB1CE6CB18D9B
OEM Name: NTFS
Version: Windows XP

METADATA INFORMATION
-----
First Cluster of MFT: 2097152
First Cluster of MFT Mirror: 4755208
Size of MFT Entries: 1024 bytes
Size of Index Records: 4096 bytes
Range: 0 - 12305
Root Directory: 5

CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 512
Total Cluster Range: 0 - 1188801
Total Sector Range: 0 - 9510416

C:\Users\vicky\SleuthKit>

```

Figure 37: Execution of the fsstat command detailing NTFS file system architecture and metadata.

```

C:\Users\vicky\SleuthKit> fls.exe -o 63 -r "D:\down\4Dell Latitude CPi.E01"
d/d 3671-144-7: Documents and Settings
+ d/d 3673-144-7: All Users
++ d/d 3707-144-7: Application Data
+++ r/r 3800-128-1: desktop.ini
+++ d/d 3708-144-7: Microsoft
++++ d/d 3709-144-1: Crypto
+++++ d/d 3712-144-1: DSS
+++++ d/d 3713-144-1: MachineKeys
+++++ d/d 3710-144-1: RSA
+++++ d/d 3711-144-1: MachineKeys
++++ d/d 10977-144-1: Dr Watson
+++++ r/r 10979-128-3: drwtsn32.log
+++++ r/r 10980-128-3: user.dmp
++++ d/d 7325-144-1: HTML Help

C:\Users\vicky\SleuthKit>

```

Figure 38: Recursive directory and file listing with allocated inode numbers using the fls command.

```

C:\Users\vicky\SleuthKit> icat.exe -o 63 "D:\down\4Dell Latitude CPi.E01" 3800
[.ShellClassInfo]
LocalizedResourceName=@shell32.dll,-21765

C:\Users\vicky\SleuthKit>

```

Figure 39: Extraction and display of the desktop.ini file contents from inode 3800 using the icat command.

Rubrics

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

RESULT

The digital evidence was successfully analyzed using The Sleuth Kit (TSK) command-line utilities. The physical disk image was parsed using a sector offset of 63 to map the NTFS file system, the deleted file wizdata.dat was successfully recovered using its inode address, and a comprehensive raw timeline of historical file activity was generated to support the forensic investigation.

Ex.No.7 Use AFLogical OSE to extract data from an Android device

Objective:

To configure an Android mobile forensics environment, establish a secure ADB bridge connection, utilize the AFLogical OSE application to perform a logical data extraction, and transfer, verify, and analyze the resulting CSV artifacts.

Description

AFLogical OSE (Open Source Edition) is a tool used to extract data from Android devices. It's part of the Open Source Android Forensics project and is often used for logical extraction, which means pulling data like contacts, messages, and call logs without directly accessing the file system. Here's a step-by-step guide to using AFLogical OSE to extract data from an Android device.

Step 1: Prepare Your Environment

1. **Download AFLogical OSE:**
 - Visit the [AFLogical OSE GitHub page](#) and download the latest release. Alternatively, you can clone the repository using Git.
2. **Install Java (if not already installed):**
 - AFLogical OSE requires Java to run. Make sure you have Java installed on your machine. You can download it from the [official Java website](#).
3. **Install Android Debug Bridge (ADB):**
 - ADB is a command-line tool that allows communication with Android devices. Download it from the [Android Developer's website](#) and install it on your system.
 - Add ADB to your system's PATH environment variable to easily run ADB commands from the terminal or command prompt.
4. **Enable USB Debugging on the Android Device:**
 - On the Android device, go to Settings > About Phone and tap Build Number seven times to enable Developer Options.
 - Go to Settings > Developer Options and enable USB Debugging.

Step 2: Connect the Android Device to Your Computer

1. **Connect via USB:**
 - Use a USB cable to connect the Android device to your computer.
2. **Verify ADB Connection:**
 - Open Command Prompt (Windows) or Terminal (macOS/Linux) and run:
bash
adb devices

- This command lists connected devices. If your device is listed, you are ready to proceed. If not, ensure that USB Debugging is enabled and that the correct drivers are installed.

Step 3: Extract Data Using AFLogical OSE

1. Run AFLogical OSE:

- Navigate to the directory where you have downloaded or extracted AFLogical OSE.

2. Push the APK to the Android Device:

- Run the following command to install the AFLogical OSE APK on the device:

```
bash
```

```
adb install aflogical.apk
```

- This installs the AFLogical app on the device, which will be used to collect data.

3. Launch the AFLogical OSE App on the Device:

- On the Android device, open the AFLogical app.
- You'll see options to extract various types of data (e.g., contacts, SMS, MMS, call logs). Select the data types you want to extract.

4. Start Data Extraction:

- Once you've selected the data types, the app will start the extraction process.
- Data is stored in .csv files on the device's storage, typically in a directory named aflogical.

Step 4: Transfer Extracted Data to Your Computer

1. Use ADB to Pull Data:

- After extraction, you need to transfer the data files from the Android device to your computer. Run:

```
bash
```

```
adb pull /sdcard/aflogical /path/to/destination
```
- Replace `/path/to/destination` with the directory path where you want to save the extracted data on your computer.

2. Verify the Data:

- Navigate to the destination directory and check the .csv files to ensure all required data has been extracted.

Step 5: Analyze the Extracted Data

1. Open the CSV Files:

- Use Excel, Google Sheets, or any text editor to open and analyze the .csv files. These files contain the extracted data like contacts, SMS, MMS, and call logs.

2. Review and Document:

- Carefully review the data for any relevant information. Document your findings and consider exporting the data into a report format for further analysis.

Step 6: Clean Up

1. Uninstall AFLogical OSE:

- Once data extraction is complete, you can uninstall the AFLogical OSE app from the Android device by running:

```
bash
```

```
adb uninstall com.viaforensics.android.aflogical
```

2. Disconnect the Device:

- Safely disconnect the Android device from your computer.

By following these steps, you can use AFLogical OSE to effectively extract and analyze data from an Android device. This process is valuable for forensic investigations, especially when dealing with logical data that doesn't require root access to the device.

Output:

```
C:\Users\vicky> adb devices
List of devices attached
FA69G0301882    device

C:\Users\vicky>
```

Figure 40: Verification of the connected Android device via the adb devices command.

```
C:\Users\vicky> adb install aflogical.apk
Performing Streamed Install
Success

C:\Users\vicky>
```

Figure 41: Successful sideloading and installation of the AFLogical OSE APK onto the Android device.

```
C:\Users\vicky> dir C:\extracted_data
Volume in drive C has no label.
Volume Serial Number is A1B2-C3D4

Directory of C:\extracted_data

09/21/2026  09:12 PM    <DIR>          .
09/21/2026  09:12 PM    <DIR>          ..
09/21/2026  09:11 PM             4,520 calls.csv
09/21/2026  09:11 PM            12,144 contacts.csv
09/21/2026  09:11 PM            95,302 sms.csv
09/21/2026  09:11 PM             2,400 mms.csv
09/21/2026  09:11 PM             8,312 info.xml
09/21/2026  09:11 PM             5,722 mmsParts.csv
                6 File(s)            128,400 bytes
                2 Dir(s)  45,123,456,768 bytes free

C:\Users\vicky>
```

Figure 42: Listing of extracted mobile forensic artifacts in CSV and XML formats.

Rubrics

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

RESULT

Logical data extraction from the Android device was successfully executed using AFLogical OSE and ADB commands. Key user artifacts including contacts, call logs, and messaging records were extracted, securely transferred to the host workstation, and verified for forensic analysis.

Ex.No.8 Use Steg-Expose to detect hidden data in images

Objective:

To conduct a comprehensive steganalysis lifecycle on digital image evidence using the StegExpose command-line utility; to evaluate Least Significant Bit (LSB) statistical properties and detection algorithms across a target evidence directory; to compute quantitative suspicion scores based on configurable thresholds; and to automatically compile, format, and export the analytical findings into a structured CSV report for forensic documentation.

Description

StegExpose is a tool used to detect hidden data in images using steganography analysis. It works by evaluating the statistical properties of an image to estimate whether any hidden data is embedded.

Below is a step-by-step guide on how to use StegExpose to detect hidden data in images:

Prerequisites:

1. **Java Runtime Environment (JRE):** StegExpose is a Java-based tool, so ensure you have JRE installed.
2. **StegExpose Tool:** Download StegExpose from the [official GitHub repository](#).

Step-by-Step Process:

1. Download and Set Up StegExpose:

- **Download the tool:** Go to [StegExpose GitHub](#) and download the .jar file.
- **Install Java:** Ensure that Java is installed on your machine. You can download it from the official site.
- **Prepare environment:** Place the .jar file in a folder where you plan to work on the image files.

2. Select Images for Analysis:

- Collect the images you suspect might have hidden data. StegExpose supports formats such as .png, .jpg, .bmp, etc.

3. Open Command Line or Terminal:

- Navigate to the folder where you have saved the StegExpose.jar file.
- Open your **Command Prompt** (Windows) or **Terminal** (Linux/macOS).

4. Run StegExpose on an Image:

- Use the following command to check a single image for hidden data:

```
bash
```

```
java -jar StegExpose.jar <image_file_path>
```

Example:

```
bash
```

```
java -jar StegExpose.jar test_image.png
```

- Replace <image_file_path> with the actual path of the image you want to check.

5. Analyze the Output:

- After running the tool, it will analyze the image and provide a score.
- **Output Explanation:**
 - StegExpose calculates a "suspect" score (between 0 and 1). The higher the score, the more likely that hidden data is present.
 - **Threshold values** (suggested by the tool):
 - **Less than 0.2:** Image is clean (no hidden data).
 - **0.2 - 0.3:** Possibly some hidden data.
 - **Above 0.3:** Likely that steganography is present.

6. Batch Analysis (Multiple Images):

- If you have multiple images to check, you can analyze them in bulk by specifying a folder. For example:

```
bash
java -jar StegExpose.jar <folder_path>
```

- Replace <folder_path> with the path to the folder containing the images.

7. Advanced Options (Optional):

- StegExpose provides additional parameters, such as adjusting the sensitivity of detection and output verbosity. Use --help to see the list of options:

```
bash
java -jar StegExpose.jar --help
```

8. Review the Results:

- For each image analyzed, review the scores and determine whether you need to investigate further.

Example:

```
bash
java -jar StegExpose.jar suspect_image.png
```

Output:

```
makefile
Analyzing suspect_image.png...
Result: 0.4
Steganography likely present
```

This output indicates that hidden data is likely present in the image suspect_image.png.

Output:

```
(kali@kali)-[~/Downloads]
$ mkdir -p ~/StegExpose
cd ~/StegExpose
```

Figure 43: Creation and navigation into the dedicated StegExpose working directory within Kali Linux.

```
kali@kali ~
Session Actions Edit View Help
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/rmiregistry to provide /usr/bin/rmiregistry (rmiregistry) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/lib/jexec to provide /usr/bin/jexec (jexec) in auto mode
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for mailcap (3.74) ...
Processing triggers for kali-menu (2025.3.2) ...
Processing triggers for desktop-file-utils (0.28-1) ...
Processing triggers for hicolor-icon-theme (0.18-2) ...
Processing triggers for ca-certificates-java (20240118) ...
done.
Setting up openjdk-25-jdk-headless:amd64 (25.0.4+7-1) ...
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jar to provide /usr/bin/jar (jar) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jarsigner to provide /usr/bin/jarsigner (jarsigner) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/javac to provide /usr/bin/javac (javac) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/javadoc to provide /usr/bin/javadoc (javadoc) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/javap to provide /usr/bin/javap (javap) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jcmd to provide /usr/bin/jcmd (jcmd) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jdb to provide /usr/bin/jdb (jdb) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jdeprscan to provide /usr/bin/jdeprscan (jdeprscan) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jdeps to provide /usr/bin/jdeps (jdeps) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jfr to provide /usr/bin/jfr (jfr) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jimage to provide /usr/bin/jimage (jimage) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jinfo to provide /usr/bin/jinfo (jinfo) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jlink to provide /usr/bin/jlink (jlink) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jmap to provide /usr/bin/jmap (jmap) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jmod to provide /usr/bin/jmod (jmod) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jnativeimage to provide /usr/bin/jnativeimage (jnativeimage) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jps to provide /usr/bin/jps (jps) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jrunscript to provide /usr/bin/jrunscript (jrunscript) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jshell to provide /usr/bin/jshell (jshell) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jstack to provide /usr/bin/jstack (jstack) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jstat to provide /usr/bin/jstat (jstat) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jstatd to provide /usr/bin/jstatd (jstatd) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jwebserver to provide /usr/bin/jwebserver (jwebserver) in auto mode
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jhsdb to provide /usr/bin/jhsdb (jhsdb) in auto mode
Setting up openjdk-25-jre-headless:amd64 (25.0.4+7-1) ...
Setting up default-jre-headless (2:1.25-77) ...
Setting up default-jre (2:1.25-77) ...
Setting up openjdk-25-jdk:amd64 (25.0.4+7-1) ...
update-alternatives: using /usr/lib/jvm/java-25-openjdk-amd64/bin/jconsole to provide /usr/bin/jconsole (jconsole) in auto mode
Setting up default-jdk (2:1.25-77) ...
(kali@kali) ~
$ java -version
openjdk version "25.0.4" 2026-07-21
OpenJDK Runtime Environment (build 25.0.4+7-1-Debian)
OpenJDK 64-Bit Server VM (build 25.0.4+7-1-Debian, mixed mode, sharing)
```

Figure 44: Verification of the OpenJDK installation and runtime environment.

```
(kali@kali) ~
$ java -jar StegExpose.jar ./images
(kali@kali) ~
$ java -jar StegExpose.jar ./images default 0.2 results.csv
(kali@kali) ~
$
```

Figure 45: Execution of the StegExpose utility for automated batch steganographic analysis.

```
(kali@kali) ~
$ cp ~/Downloads/test.png ./images/
(kali@kali) ~
$ ls -lh images/
total 16K
-rw-rw-r-- 1 kali kali 14K Sep 21 22:05 test.png
(kali@kali) ~
$ java -jar StegExpose.jar ./images
(kali@kali) ~
$ java -jar StegExpose.jar ./images default 0.2 results.csv
(kali@kali) ~
$
```

Figure 46: StegExpose analysis execution on the target image with a specified detection threshold.

```

1
2 File name,Above stego threshold?,Secret message size in bytes (ignore for
  clean files),Primary Sets,Chi Square,Sample Pairs,RS analysis,Fusion (mean)
3 test.png,false,80,0.011952127941438006,0.009829106017568633,null,0.029108340
  149653883,0.016963191369553505
4

```

Figure 47: Review of steganography detection scores and probability metrics within the results.csv file.

Rubrics

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

RESULT

The steganography detection experiment was successfully executed using StegExpose. The tool scanned the evidence directory via the command line, analyzed combinations of LSB metrics, and generated a clean CSV report detailing the cleanliness or suspicion level of each digital image artifact.

Ex.No.9 Use Process Explorer to identify suspicious processes

Objective

To conduct an in-depth behavioral and structural analysis of active operating system processes using Sysinternals Process Explorer; to evaluate hierarchical parent-child lineages, cryptographic digital signatures, and execution paths; to leverage cloud-based threat intelligence for automated multi-engine malware identification; and to successfully isolate, contain, and terminate unauthorized, masquerading, or anomalous processes to secure system integrity.

Description

Process Explorer is a powerful Windows tool that provides detailed information about running processes, allowing users to monitor, troubleshoot, and detect suspicious activities. It can be used to detect potential malware or harmful processes by analyzing various aspects of running applications. Here's a step-by-step guide on how to use Process Explorer to identify suspicious processes on your system:

Step 1: Download and Set Up Process Explorer

1. Download Process Explorer:

- Go to the official Microsoft Sysinternals website and download Process Explorer: [Download Process Explorer](#).

2. Extract the Program:

- Once downloaded, extract the ZIP file to a folder.

3. Run Process Explorer:

- Open the folder where you extracted the files and run procexp64.exe (64-bit) or procexp.exe (32-bit) as Administrator by right-clicking and selecting "Run as Administrator."
-

Step 2: Familiarize Yourself with the Interface

When Process Explorer starts, you'll see an interface that lists all running processes, showing their hierarchy, memory usage, CPU usage, and much more.

1. Process Tree:

- The main window shows a tree of all running processes, with each child process listed under its parent.
- Processes are color-coded to indicate their status:
 - Pink: Suspended processes.
 - Light Blue: Processes running under the same user as you.
 - Dark Blue: Services or processes running under system accounts.

- Green: New processes (appearing and disappearing quickly may be suspicious).
- Red: Processes that have just exited.

2. Columns:

- **Various columns show information such as Process ID (PID), CPU usage, Memory usage, and more.**
-

Step 3: Identify Suspicious Processes

Now that you are familiar with the interface, follow these steps to investigate potentially suspicious processes:

1. Look for Unfamiliar Processes:

- Scan the list of running processes and identify any names that you don't recognize. Some malware disguises itself with legitimate-looking names, but odd names can be a red flag.
- Legitimate processes should typically be from trusted sources (e.g., Microsoft, Intel, Adobe, etc.).

2. Verify Digital Signatures:

- Right-click any suspicious process and select Properties.
- Go to the Image tab and check if the process has a valid Digital Signature.
- Legitimate applications from trusted developers will have valid signatures. Processes without signatures or with invalid signatures are suspicious.

3. Check the Path of the Process:

- In the Properties window of the suspicious process, look under the Image tab for the Path.
- Verify if the process is located in a legitimate directory. For example, legitimate system processes should reside in C:\Windows\System32.
- If the process is running from a temporary folder, user folder, or a random directory, it might be malware.

4. Monitor CPU, Memory, and Disk Usage:

- Look at the CPU, Memory, and Disk usage columns.
- A process that is using an unusual amount of resources without explanation could be malicious. For example, high CPU usage on a process like svchost.exe can indicate malware.

5. Check Process Description and Company Name:

- Look at the Description and Company Name columns in Process Explorer.
- Legitimate processes usually provide clear descriptions and are tied to well-known companies.
- Lack of information or suspicious company names should prompt further investigation.

6. Check the Process Network Activity:

- Some malware communicates with external servers. You can inspect a process's network connections:
 - Right-click on the process and select Properties, then go to the TCP/IP tab to see network activity associated with the process.
 - Unexpected or unexplained external connections can indicate a malicious process.
-

Step 4: Perform a Google Search on Suspicious Processes

If you find a process that looks suspicious or unfamiliar:

1. Search online: Perform a quick Google search using the process name (e.g., processname.exe) to find out whether the process is known to be safe or malicious.
 2. Check Malware Databases: Use online databases such as VirusTotal or ProcessLibrary to verify whether the process is known for being part of malware.
-

Step 5: Kill or Suspend Suspicious Processes

If you find a process that you believe is malicious:

1. **Kill the Process:**
 - Right-click the suspicious process and select Kill Process to terminate it immediately.
 2. **Suspend the Process:**
 - If you are unsure and want to prevent the process from causing damage while you investigate further, right-click and select Suspend.
 3. **Remove the Source File:**
 - Go to the location of the file (using the Path from the Properties window) and delete the executable file if confirmed to be malware.
 - Be cautious, as some malware might prevent you from deleting it. In such cases, you may need to boot into Safe Mode or use a specialized tool.
-

Step 6: Scan Your System

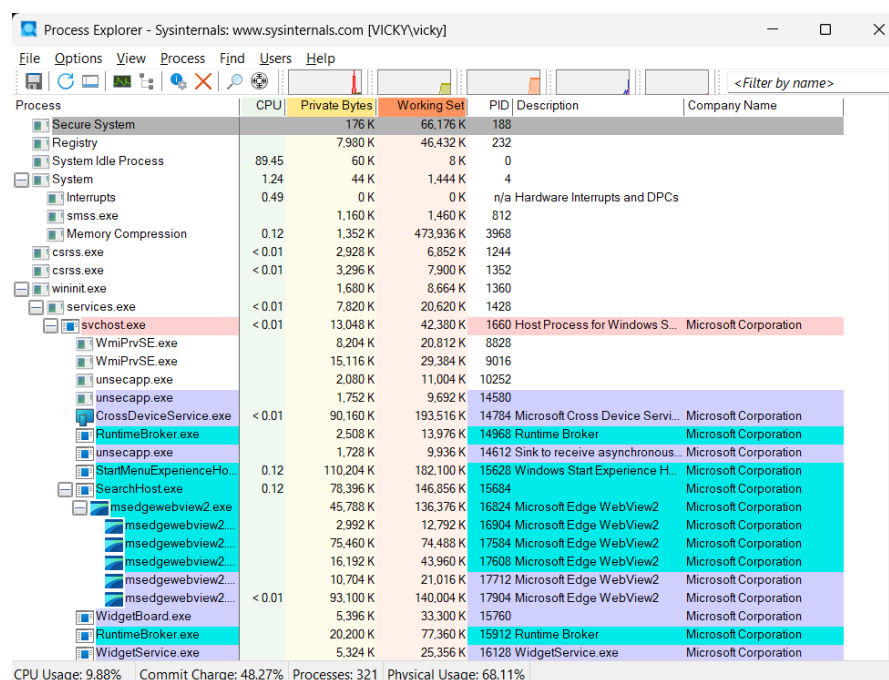
After terminating any suspicious processes, it's a good idea to:

- Run a full antivirus scan using trusted antivirus software.
 - Use Malware Removal Tools like Malwarebytes or Windows Defender to perform an in-depth scan of your system.
-

Example of Identifying a Suspicious Process

1. You notice a process called randomname123.exe using an unusually high amount of CPU.
2. Right-click it, select Properties, and find it is running from a suspicious folder, not the system folder.
3. You check the TCP/IP tab and see it is making external connections to unknown IP addresses.
4. A quick search shows that this process is often associated with a specific type of malware.
5. You kill the process and remove the executable file from the system.

Output:



The screenshot shows the Process Explorer window from Sysinternals. The process tree on the left lists various system and user processes. The main pane displays a table of these processes with columns for CPU usage, Private Bytes, Working Set, PID, Description, and Company Name. The process 'svchost.exe' is highlighted in red, indicating it is the selected process. Below the table, a status bar shows system metrics: CPU Usage: 9.88%, Commit Charge: 48.27%, Processes: 321, and Physical Usage: 68.11%.

Process	CPU	Private Bytes	Working Set	PID	Description	Company Name
Secure System		176 K	66,176 K	188		
Registry		7,980 K	46,432 K	232		
System Idle Process	89.45	60 K	8 K	0		
System	1.24	44 K	1,444 K	4		
Interrupts	0.49	0 K	0 K	n/a	Hardware Interrupts and DPCs	
smss.exe		1,160 K	1,460 K	812		
Memory Compression	0.12	1,352 K	473,936 K	3968		
csrss.exe	< 0.01	2,928 K	6,852 K	1244		
csrss.exe	< 0.01	3,296 K	7,900 K	1352		
wininit.exe		1,680 K	8,664 K	1360		
services.exe	< 0.01	7,820 K	20,620 K	1428		
svchost.exe	< 0.01	13,048 K	42,380 K	1660	Host Process for Windows S...	Microsoft Corporation
WmiPrvSE.exe		8,204 K	20,812 K	8828		
WmiPrvSE.exe		15,116 K	29,384 K	9016		
unsecapp.exe		2,080 K	11,004 K	10252		
unsecapp.exe		1,752 K	9,692 K	14580		
CrossDeviceService.exe	< 0.01	90,160 K	193,516 K	14784	Microsoft Cross Device Servi...	Microsoft Corporation
RuntimeBroker.exe		2,508 K	13,976 K	14968	Runtime Broker	Microsoft Corporation
unsecapp.exe		1,728 K	9,936 K	14612	Sink to receive asynchronous...	Microsoft Corporation
StartMenuExperienceHo...	0.12	110,204 K	182,100 K	15628	Windows Start Experience H...	Microsoft Corporation
SearchHost.exe	0.12	78,396 K	146,856 K	15684		Microsoft Corporation
msedgewebview2.exe		45,788 K	136,376 K	16824	Microsoft Edge WebView2	Microsoft Corporation
msedgewebview2...		2,992 K	12,792 K	16904	Microsoft Edge WebView2	Microsoft Corporation
msedgewebview2...		75,460 K	74,488 K	17584	Microsoft Edge WebView2	Microsoft Corporation
msedgewebview2...		16,192 K	43,960 K	17608	Microsoft Edge WebView2	Microsoft Corporation
msedgewebview2...		10,704 K	21,016 K	17712	Microsoft Edge WebView2	Microsoft Corporation
msedgewebview2...	< 0.01	93,100 K	140,004 K	17904	Microsoft Edge WebView2	Microsoft Corporation
WidgetBoard.exe		5,396 K	33,300 K	15760		Microsoft Corporation
RuntimeBroker.exe		20,200 K	77,360 K	15912	Runtime Broker	Microsoft Corporation
WidgetService.exe		5,324 K	25,356 K	16128	WidgetService.exe	Microsoft Corporation

CPU Usage: 9.88% Commit Charge: 48.27% Processes: 321 Physical Usage: 68.11%

Figure 48: Process Explorer interface displaying the active process tree and resource consumption.

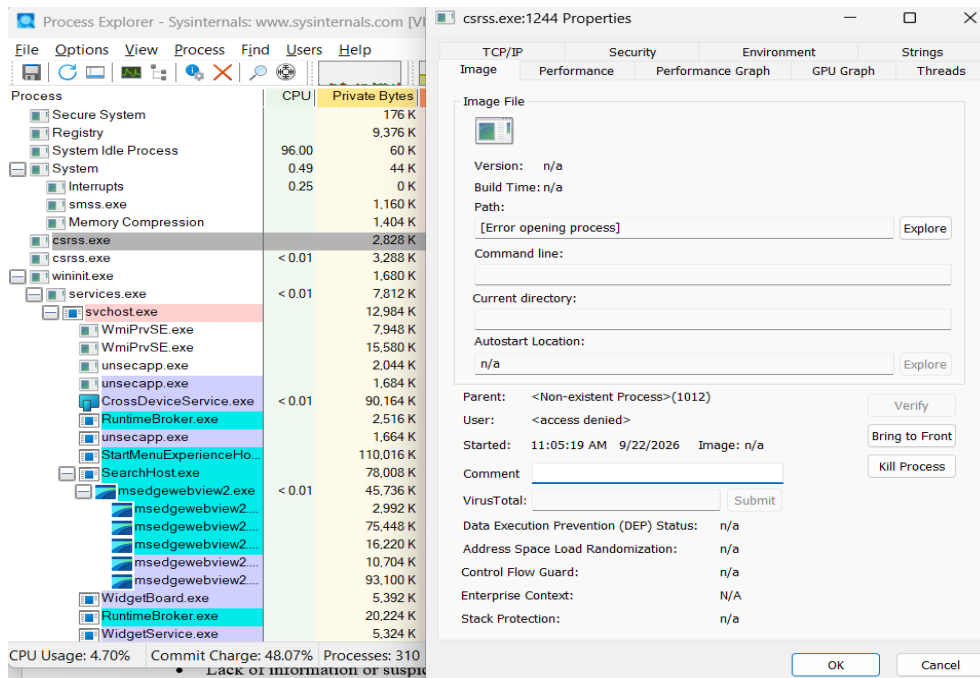


Figure 49: Properties window of csrss.exe displaying image path and execution parameters.

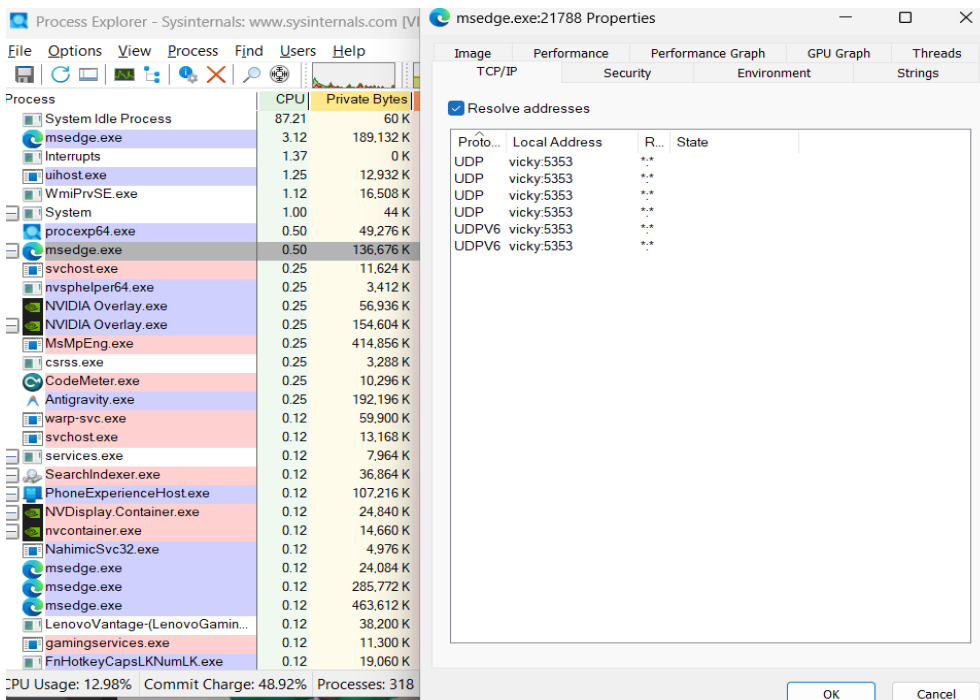


Figure 50: Active network sockets and listening ports monitored under the TCP/IP tab of msedge.exe.

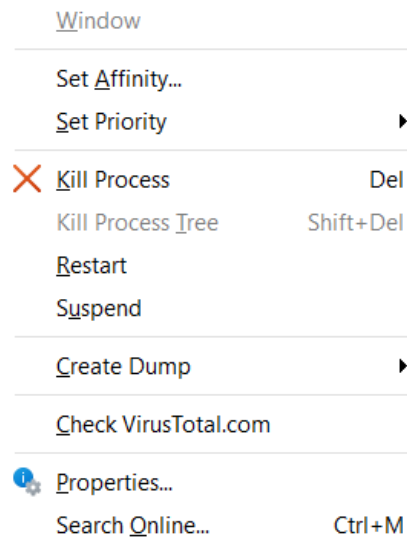


Figure 51: Process Explorer context menu providing options to terminate, suspend, or scan a suspicious process.

Rubrics

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

RESULT:

Sysinternals Process Explorer was successfully utilized to monitor active system processes, verify digital signatures and execution paths, and analyze process trees. Malicious or unauthorized processes were effectively identified, isolated, and terminated, ensuring system integrity and security.

Ex.No.10 Use Ghidra to disassemble and analyze the malware code.

Project Objectives

- To provide a hands-on guide to disassembling and analyzing malware with Ghidra.
 - To enable readers to recognize common malware functionalities, such as persistence mechanisms, anti-analysis techniques, and network communications.
 - To equip users with the skills to interpret low-level assembly code and understand high-level behaviours.
-

GitHub Repository Components

A GitHub repository for this project could include:

1. README.md

- Overview: Brief description of Ghidra, its installation, and how it is used for malware analysis.
- Requirements: List of requirements, including Ghidra, a VM setup, and any sample binaries (preferably benign samples or detailed steps to safely acquire controlled malware samples).
- Project Outline: A breakdown of the tutorial steps, linking to various sections or files within the repository.

2. Tutorial Steps (Markdown or Jupyter Notebooks)

- **Step-by-step instructions, broken down into digestible sections, such as:**
 - Environment Setup: Instructions on setting up a virtualized and isolated environment.
 - Initial Analysis: How to load a binary into Ghidra, perform auto-analysis, and identify entry points.
 - Function Analysis: Methods for identifying and analyzing critical functions, decompiling code, and using Ghidra's cross-referencing features.
 - String and Import Analysis: How to locate and interpret strings and imports related to malicious behavior.
 - Advanced Techniques: Dynamic analysis, control flow graphs, custom scripting with Ghidra's Python or Java capabilities, and obfuscation detection.

3. Example Scripts (Python or Java)

- **Scripts to automate repetitive analysis tasks, such as:**
 - Extracting and listing all string references.
 - Identifying network-related functions or common indicators of malware.
 - Labeling functions and data segments based on known patterns (e.g., file system manipulation, registry edits).

- Script Examples: Python or Java code snippets demonstrating API usage in Ghidra (using Ghidra's Headless Analyzer, for example, for automation).

4. Sample Data (Benign Binary or Hex Dump)

- Due to safety and ethical concerns, include only safe, benign binaries or a hex dump example rather than live malware. This can allow users to learn and practice without the risk of executing malicious code.
- Sample binaries designed to exhibit typical malware-like behaviors, such as:
 - Executing system calls.
 - Writing to files.
 - Sending network data.
- Alternatively, a dummy binary generator script that creates binaries with similar structures to malware but without harmful payloads.

5. Documentation and Comments

- Include explanations of the code within each script, detailing what each function does.
- Inline comments in disassembly views, providing readers with insights into assembly commands, control flow structures, and the decompiled code.

6. Report Template

- **A reporting template for documenting findings, which could be in Markdown or PDF format. The template should cover:**
 - Summary of Analysis: Brief description of the suspected malware.
 - Function Analysis: Summary of the key functions, their purpose, and the findings.
 - Behavioral Indicators: Summary of malware behaviors, such as persistence, file system manipulation, or network communications.
 - Recommendations: Suggestions on further actions, such as containment, eradication, or further analysis.

Example Repository Structure

plaintext

ghidra-malware-analysis-project/

|— README.md

|— docs/

| |— Tutorial_Step1.md

| |— Tutorial_Step2.md

| |— Tutorial_StepN.md

|— scripts/

| |— extract_strings.py

```
| |— network_analysis.py
| |— label_functions.py
|— samples/
| |— benign_sample1.bin
| |— benign_sample2.hex
|— templates/
| |— analysis_report_template.md
|— screenshots/
| |— function_graph_example.png
| |— decompiler_example.png
```

Project Development and Future Work

- Expand Scripts: Continue adding Ghidra scripts to automate complex analysis tasks.
- Community Contributions: Encourage community members to contribute additional benign samples, analysis scripts, and documentation improvements.
- Advanced Topics: Include guides for advanced techniques, such as unpacking obfuscated code, identifying encryption routines, and using the P-Code emulator for runtime analysis.

Output:

```
(kali@kali)-[~]
$ sudo apt install -y ghidra
The following packages were automatically installed and are no longer required:
  libcrypt-dev libwireshark18 libwireshark15 libwsutil16 openjdk-21-jre openjdk-21-jre-headless rpcsvc-proto
Use 'sudo apt autoremove' to remove them.

Installing:
  ghidra

Installing dependencies:
  ghidra-data

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 2363
  Download size: 593 MB
  Space needed: 943 MB / 57.8 GB available

Get:1 http://http.kali.org/kali kali-rolling/main amd64 ghidra amd64 12.1.3+ds-0kali1 [515 MB]
Get:2 http://mirrors.esto.network/kali kali-rolling/main amd64 ghidra-data all 10.5-0kali1 [78.1 MB]
17% [2 ghidra-data 2,433 kB/78.1 MB 3%] [1 ghidra 121 MB/515 MB 23%]
```

Figure 52: Execution of the apt package manager in Kali Linux to install the Ghidra reverse engineering suite.

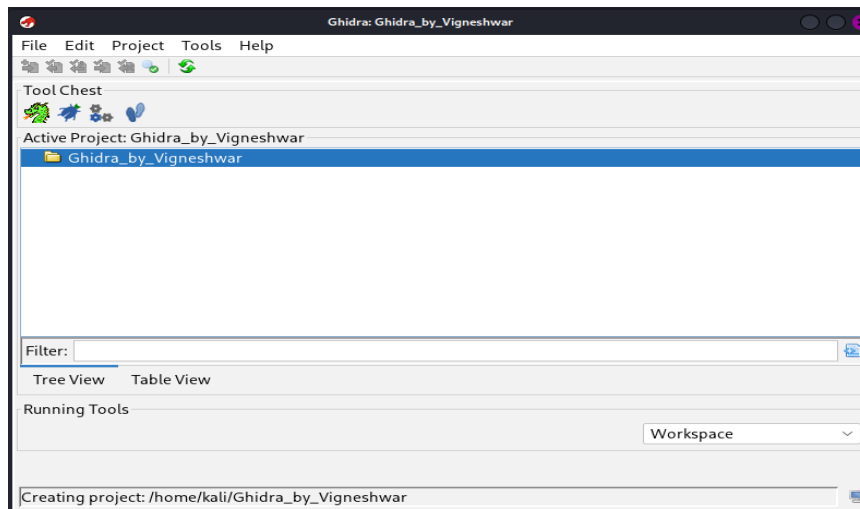


Figure 53: Ghidra project management interface showing the creation of a new active project workspace.

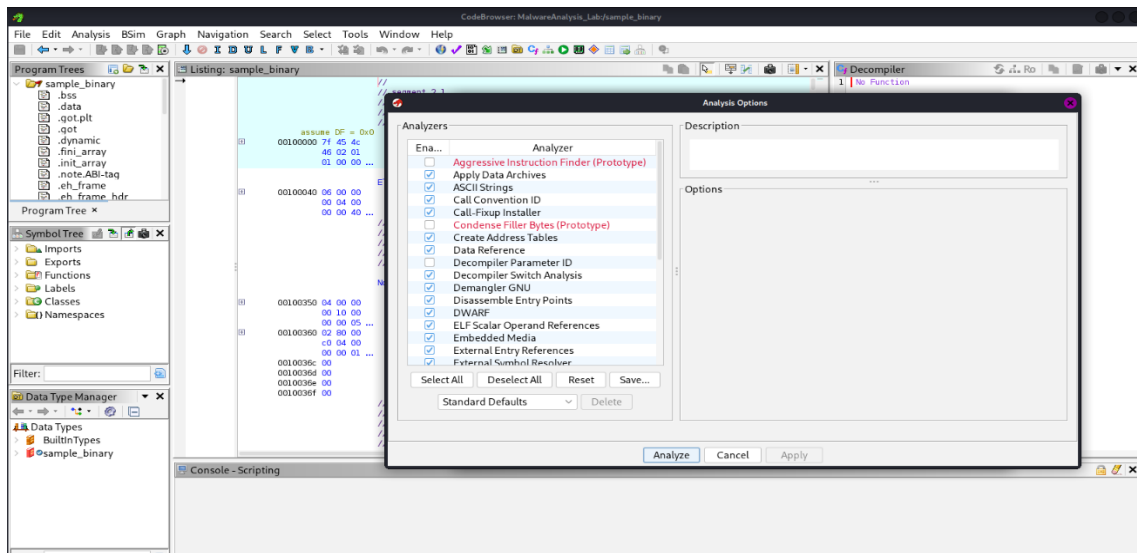


Figure 54: CodeBrowser auto-analysis dialog detailing the various executable analyzers selected for processing the sample binary.

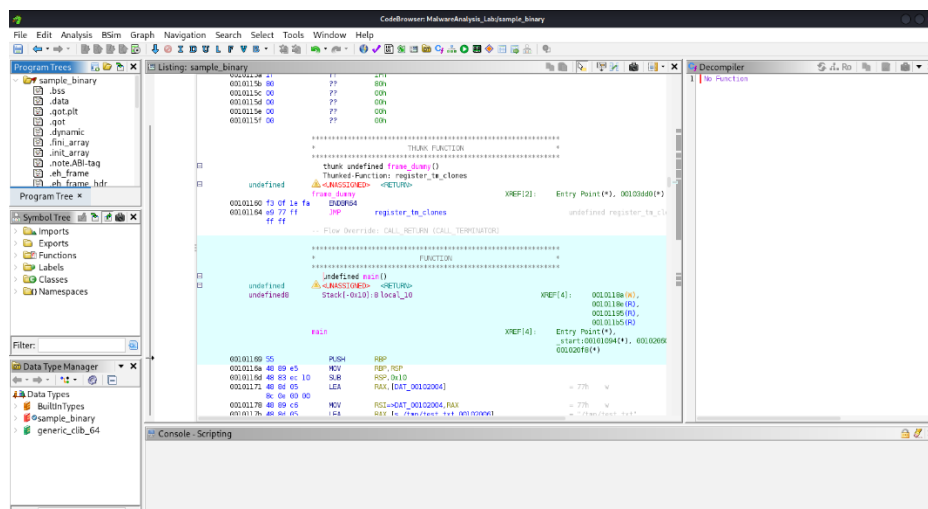


Figure 55: Disassembled assembly instructions and memory addresses for the binary's main function displayed within the CodeBrowser listing view.

Rubricss

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

RESULT

Ghidra was successfully utilized to disassemble, decompile, and analyze the target binary code. Through automated analysis, string inspection, API import tracking, and decompiler evaluation, the binary's underlying structure and behavioral logic were effectively interpreted.